Efficient Reinforcement Learning for Small Formal Reasoners

Experiment record and formalization, 1 to 8 October 2026

Manas Ravulapalli
Compiled from the project repository and run records

12 October 2026

Abstract

This report records every experiment of the project from 1 to 8 October 2026, with the formal definitions behind them. The project asks whether reinforcement learning (RL) can teach `Qwen2.5-0.5B-Instruct` number theory and Lean 4 proofs under a fixed compute budget. We ran 13 experiments: data preparation, two capability baselines, three proof-RL studies with a terminal Lean reward, five knowledge-acquisition studies with an RLP reward, one evaluation series of saved checkpoints, and one RL pilot with a Lean-verified local reward. In these early experiments, the model never produced an accepted Lean proof without assistance. Lean accepted 0 of 63 whole-proof attempts in the recounted local check, and the exported records of a larger sampling run show 0 of 273. RL with a terminal Lean reward gave no learning signal, because no sampled group had reward variation. In one RLP run, the prediction reward rose while held-out accuracy fell from 42/108 to 21/108. The only measured RL gain in this early record came from a Lean-checked witness reward on a narrow task, where validation accuracy rose from 0/8 to 6/8. A post hoc analysis shows that none of the 63 failed proof attempts was well-formed Lean 4. An audit of the Lean scorer on 9 and 10 October found nine kinds of input that it accepted and should have rejected, and a fix rejects all nine. A single tactic from a fixed list proves 28 of 420 validation statements, so a reward for accepted proofs can rise without proof skill. At the scorer-audit snapshot, the Lean GRPO trainer had not run on a GPU; that statement predates the later training in Section 1.2. The report ends with a proposed study that has not run.

Status of this document

The two-page companion *Lean-in-the-loop RL: setup and verification boundaries* (source: `docs/tex/training_setup.tex`) describes the prepared setup and ordered next work as of 11 October 2026. It separates step verification, private-target likelihood rewards, and complete-proof evaluation. That companion describes preparation at its stated date; it is not the current execution record. The historical sections below retain their original scope through 8 October. Section 1.2 distinguishes subsequent GRPO training from the interactive REINFORCE and prepared PPO comparisons. The abstract's statements about absent unassisted proofs and absent GPU training describe the early studies and scorer-audit snapshot. They do not describe the later interleaved and recovery checkpoints. Current execution status remains in `README.md`, with protocols and evidence indexed in `docs/EXPERIMENTS.md`. Appendix C adds the dated analysis of the recent Lean loop and its failure boundaries. Its unfinished checks are marked pending; it does not replace the historical results.

- Every count comes from a run record. Appendix B names the record and the raw artifact for each experiment.

- The detailed run records are archived in Git commit `8fc8423` under `docs/results/` and `docs/decisions.md`. Each of those records names the CPU recount script that produced its counts.
- For this report, three independent checks re-read 89 counts of Experiments 1, 2, and 4 to 13 from the raw artifacts, and all 89 matched. We re-read the counts of Experiment 3 and Section 9 directly. Appendix B lists the sources.
- Section 9 reports two analyses that we ran on 8 October 2026, after all training runs. Their scripts are in `scripts/audit/`.
- Section 10 reports an audit of the Lean reward that we ran on 9 and 10 October 2026. It adds no experiment to Table 1. Its counts come from run reports on cpu-angad. Nobody re-read them from the raw files.
- The experiment descriptions in the formal terms of Section 3 were written after the runs. They describe the designs. They do not reinterpret the results.
- Paths name the machine and a relative location, for example cpu-angad `formal_rl/artifacts/` or S3 `capstate/acq/formal_rl/`. Bucket names, addresses, and instance identifiers are kept out of the repository.

Contents

1 Introduction **4**
- 1.1 Question and scope . . . 4
- 1.2 GRPO lineage and the current comparison . . . 5
- 1.3 Findings in brief . . . 6

2 Setup **6**
- 2.1 Model . . . 6
- 2.2 Lean environment and the checker . . . 8
- 2.3 Compute and accounting . . . 8
- 2.4 Grading . . . 8

3 Formalization **9**
- 3.1 Capability decomposition . . . 9
- 3.2 Mathematical records . . . 9
- 3.3 Information views . . . 10
- 3.4 Learning signals . . . 10
- 3.5 Curriculum formalism . . . 11
- 3.6 Evaluation formalism . . . 12

4 Data and capability baselines **13**
- 4.1 Experiment 1: LeanDojo subset and number-theory pool . . . 13
- 4.2 Experiment 2: Base-model math probe and GSM8K LoRA . . . 14
- 4.3 Experiment 3: Instruct proof check . . . 14

5 Proof RL with a terminal Lean reward **15**
5.1 Experiment 4: Whole-proof GRPO . . . 15
5.2 Experiment 5: Prerequisite curriculum GRPO and the syntax retry . . . 15
5.3 Experiment 6: Assisted proof completion . . . 17

6 Knowledge acquisition with the RLP reward **17**
6.1 Experiment 7: RLP training and primitive-first continuation . . . 17
6.2 Experiment 8: Precision audit and FP32 rerun . . . 18
6.3 Experiment 9: Primitive RLP curriculum . . . 18
6.4 Experiment 12: Audit of the primitive RLP corpus . . . 19
6.5 Experiment 13: Controlled RLP, ordered arm . . . 20

7 Evaluations of saved checkpoints (Experiment 10) **20**
7.1 Content probe . . . 20
7.2 Acquisition, proof semantics, and proof strategy . . . 21
7.3 Knowledge inventory and Lean witness controls . . . 21

8 RL with a Lean-verified local reward (Experiment 11) **22**

9 Post hoc analyses, 8 October 2026 **22**
9.1 Why the Instruct proof attempts failed . . . 22
9.2 Material for a proof-state curriculum . . . 23

10 Audit of the Lean reward, 9 and 10 October 2026 **23**
10.1 What the trainer pays . . . 25
10.2 Inputs that the scorer accepted without a proof . . . 25
10.3 Proofs that the project does not want paid . . . 28
10.4 Valid proofs of statements without information . . . 28
10.5 Valid proofs that the policy can recall . . . 29
10.6 Weaker checks that accept a non-proof . . . 30
10.7 Reward that covers part of the output, and reward noise . . . 31
10.8 Checks before an RL gain counts, and open items . . . 31

11 What the record shows **32**
11.1 Threats to validity . . . 33

12 Proposed next study (not run) **33**

A Training settings **35**

B Provenance **35**

C Current Lean loop and failure analysis, 13 October **37**
C.1 The current experiment uses Lean during RL . . . 37
C.2 Reset and transition contracts are implemented, but fresh checks remain pending . . . 37
C.3 RL changes intermediate behavior without improving the measured endpoint . . . 38
C.4 Repeated failures do not have one tactic as their explanation . . . 39
C.5 Lost diagnostics can change the intended feedback signal . . . 39
C.6 Uniform occurrence does not provide a complete prerequisite curriculum . . . 40

C.7 Proof delivery has a separate serialization defect . . . 40
C.8 Evidence boundaries and next checks . . . 40
C.9 Environment repairs after the source audit . . . 41
C.10 Published small provers guide interface design, not predicted success . . . 41
C.11 The next causal study remains a separate experiment . . . 41

1 Introduction

1.1 Question and scope

The project asks when RL improves number-theory knowledge and Lean 4 proof construction in a model with about 500 million parameters, under a fixed compute budget. The midsemester goal is a model that generates some verified proofs. The longer goal is proof strategy on competition problems with RL in the loop. Neither goal was met by the early experiments below. Subsequent proof demonstrations have a narrower scope than the longer goal.

This report covers 13 experiments. Table 1 lists them. The numbers match `docs/EXPERIMENTS.md`. The sections group the experiments by the question they answer.

Midterm question adopted on 11 October. Can verified demonstrations and solvable training tasks make RL improve our small prover? This question remains open. The current experiment status is maintained in `README.md`; the historical results below do not establish this claim.

A decisive comparison starts from the same checkpoint after supervised learning on Lean-verified training proofs. Compare continued supervised learning with GRPO on training tasks that produce both accepted and rejected attempts. Match the training compute budget and use identical evaluation settings. Select tasks and checkpoints without using held-out test outcomes.

The primary endpoint is complete, unassisted Lean-accepted proofs on unseen theorems. Report generated tokens and Lean calls beside proof coverage. Assisted completions, valid intermediate tactics, and improved training reward do not establish this endpoint. The existing EXP-021 recovery protocol tests an added RL stage, but does not isolate RL from all additional training effects. Its limits remain part of the interpretation.

Current failure diagnosis, 11 October. EXP-021's frozen DEV recount finds no accepted proof in any arm. Each arm contains 64 attempts across 32 statements. In the RFT-plus-GRPO arm, the first errors classify as 43 proof-construction failures, 17 name or type-resolution errors, 3 syntax or structure errors, and 1 verifier timeout. The shared starting checkpoint has 41 proof-construction failures, 18 name or type-resolution errors, 4 syntax or structure errors, and 1 timeout. These are exclusive first-error locations, not causes of failed learning or judgments of informal mathematical reasoning. The separate tactic pilots rejected fenced actions before Lean execution. Their parser correction does not establish corrected policy training.

The recount uses `scripts/audit/classify_proof_failures.py` on raw `checked.jsonl` records. Its summaries and source snapshot reside in the EXP-021 `proof_failure_diagnostic/` archive indexed by `docs/EXPERIMENTS.md`. EXP-022 prepares a PPO versus GRPO comparison from the same parent and with matched sampling. Its CPU checks establish adapter mechanics; proof improvement remains untested.

1.2 GRPO lineage and the current comparison

Group Relative Policy Optimization (GRPO) compares sampled responses to the other responses for the same prompt [3]. The reward defines what this comparison teaches. The project uses this optimizer for both hidden-answer likelihood rewards and Lean-verified rewards; these objectives certify different things. Equation (8) explains why equal rewards remove the task-reward policy gradient. This diagnosis applies to the early terminal-reward failures, not to every later checkpoint.

Whole proofs and prerequisites. Experiments 4 and 5 start from the original Instruct model and request complete proof bodies. The prerequisite study adds authored logic, arithmetic, divisibility and induction tasks, with decreasing support and a syntax-template retry. The sampled rewards provide no contrast, so these studies cannot establish an effect of curriculum order. Experiment 6 supplies a proof prefix and creates occasional accepted completions. Its lack of unassisted validation success prevents a claim that finishing a supplied proof transfers to constructing one. Their methods and historical counts remain in Section 5.

Local mathematical witnesses. Experiment 11 starts from the primitive-RLP checkpoint and asks for an expression satisfying a supplied mathematical obligation. Lean certifies that expression, rather than a complete proof or the surrounding explanation. The local result in Section 8 therefore measures witness generation. It does not establish complete-proof competence.

Interleaved proof training. EXP-020 starts from Lean SFT v2, adds a common supervised stage, then separates SFT/RLP and SFT/Lean-GRPO branches. The GRPO schedule begins with short proof-prefix completions, allows longer continuations in later stages, and finally includes whole-proof tasks. Supervised refreshes occur between RL stages. The terminal checkpoints are evaluated without those training prefixes. The catalogue's validation recount reports accepted complete proofs and concentration on the continuation `simp_all` in the GRPO branch. It also records lower terminal GRPO completion than the common supervised parent on the primary population. These are saved verifier outcomes, not a fresh Lean replay. Additional supervised stages and different branch data prevent attribution of the comparison to the RL algorithm alone. The cause of the output concentration remains untested.

Recovery before another GRPO stage. EXP-021 retains the common supervised parent and compares continued SFT, rejection-sampling fine-tuning, and that fine-tuning followed by KL-regularized GRPO. Rejection-sampling fine-tuning means supervised training on generated training proofs that Lean accepts. The catalogue records completed recovery checkpoints but no accepted proof in its frozen DEV recount, including the shared starting checkpoint. Its first-error classifications locate failures; they do not establish why training failed. The comparison tests an added training stage and does not isolate the KL penalty or all effects of continued training.

Interactive RL and PPO are separate methods. The stepwise EXP-019 implementation uses on-policy REINFORCE with reward-to-go and a leave-one-out baseline. The policy samples tactics, receives Lean feedback, and continues from the accepted proof state. Grouping trajectories does not make this implementation GRPO. Its later narrow proof demonstrations do not establish undergraduate coverage or held-out generalization. EXP-022 prepares a matched whole-proof PPO/GRPO comparison. EXP-023 instead prepares interactive REINFORCE reward and curriculum comparisons, alongside a matched one-action PPO/REINFORCE comparison. Its latest GPU attempt completes supervised training but fails in the discarded PPO pilot before any RL arm runs.

CPU checks of the repair do not establish full-model GPU execution or proof improvement. The recovery remains prepared and unlaunched at this report update.

Evidence locations and presentation boundary. The later protocols, raw-outcome locators, failure records and recount commands are indexed under EXP-019 to EXP-023 in `docs/EXPERIMENTS.md`. EXP-020 uses `configs/midtrain/qwen_05b_interleaved_training_v3.json` and `interleaved_eval_20261010_v4/`. EXP-021 uses `configs/midtrain/qwen_05b_monday_recovery.json` and its `proof_failure_diagnostic/` archive. The interactive entry point is `scripts/train/run_tactic_rl.py`; the whole-proof entry point is `scripts/train/run_lean_grpo.py`. This lineage update introduces no new numerical result or independent verifier certification. GRPO execution, accepted proofs and improvement over a matched parent are separate claims.

1.3 Findings in brief

1. A terminal Lean reward gave no learning signal while the model never succeeded. This blocked Experiment 4, Experiment 5, and most of Experiment 6.
2. The unchanged model does not write well-formed Lean 4. In the 63 failed attempts of Experiment 3, 57 had a parse or file-structure error, 3 were empty, and 3 used `sorry` (Section 9).
3. A Lean-checked reward worked once the task was small enough for the model to succeed sometimes (Experiment 11).
4. The RLP reward can rise while correctness falls (Experiment 9). Correct training targets did not make the generated reasoning correct.
5. No curriculum experiment produced a scored comparison of ordered against shuffled training.
6. AI graders disagreed on 44 of 321 response views in Experiment 10. Correctness scores without a checker are noisy.
7. Before a fix on 10 October, the scorer of the Lean GRPO trainer accepted 27 of 87 attack checks. After the fix, it accepted 0 of 87 (Section 10).
8. A single tactic from a fixed list proves 161 of 3,450 training statements and 28 of 420 validation statements. Another 17, 3, and 3 statements in the train, validation, and test splits have hypotheses that prove `False` (Section 10).

2 Setup

2.1 Model

Every experiment used Qwen2.5 with 0.5 billion parameters [1]. Experiment 2 used the base model. All other experiments used `Qwen/Qwen2.5-0.5B-Instruct` or a checkpoint trained from it. We read Table 2 from the model configuration and the header of the weight file. The parameter total matches the count in the weight audit of Experiment 8.

The Qwen2.5 report states that pretraining used 18 trillion tokens, and that post-training used supervised fine-tuning on more than one million samples followed by multistage RL [1]. The report does not state how much Lean 4 the data contains. The tokenizer encodes each of the Lean symbols $\vdash$, $\mathbb{N}$, $|$, $\equiv$, $\forall$, $\exists$, $\langle$, $\rangle$, $\to$, and $\uparrow$ as one token. It splits mathlib names into pieces; `Nat.Prime` takes three tokens.

Table 1: All experiments, 1 to 6 October 2026. "Start" is the policy that each run started from. "ck-118" is `checkpoint-118` of Experiment 9. A count n/m is n successes in m items.

#	Experiment	Date	Start	Main result
1	LeanDojo data subset and number-theory pool	10-01	–	706/17/15 Nat theorems; 2,658/43/38 broader pool. Lean replay not done
2	Base-model math probe and GSM8K LoRA	10-01	Base	GSM8K 9/24 before, 8/24 after LoRA. MATH number theory 3/20 both
3	Instruct proof check, local and GPU sampling	10-01, 10-03	Instruct	Lean accepted 0/63 local attempts. GPU sampling records show 0/273
4	Whole-proof GRPO, random vs. family order	10-03	Instruct	0/8 validation theorems in every phase. No group had reward variation
5	Prerequisite curriculum GRPO and syntax retry	10-04	Instruct	0/256 sampled proofs accepted. GRPO skipped
6	Assisted proof completion	10-04	Instruct	2/64 with a missing expression. One theorem qualified. Unassisted held-out 0/16
7	RLP number-theory training and continuation	10-04	Instruct	Held-out 11/48 before and after the continuation. Validation 7/16 throughout
8	Precision audit and FP32 rerun	10-04	RLP parent	8,557,971 of 494,032,768 weights changed. FP32 rerun: held-out 11/48, validation 7/16 to 6/16
9	Primitive RLP curriculum	10-05	Instruct	Held-out fell from 42/108 to 21/108. Validation peaked at 25/56 at ck-118
10	Evaluations of saved checkpoints	10-05	–	Early RLP: explained recognition 14/32 vs. 7/32. No gain in complete applications or proofs
11	Local-action pilot, Lean-verified reward	10-06	ck-118	Validation 0/8 to 6/8; held-out 0/4 to 2/4. Error-free answers 2/8 and 0/4
12	RLP corpus audit	10-06	–	All 612 numeric targets correct. 160 of 448 training rules lack domain or zero conditions
13	Controlled RLP, ordered arm	10-06	ck-118	0/18 fully correct at all five checkpoints. Shuffled arm not scored

Table 2: Architecture of `Qwen2.5-0.5B-Instruct`, read from `config.json` and `model.safetensors`.

Property	Value
Class	`Qwen2ForCausalLM`, decoder-only transformer
Parameters	494,032,768, of which 136,134,656 (27.6%) are the embedding table
Layers	24, each with 14,912,384 parameters
Hidden size	896
Attention	Grouped-query: 14 query heads, 2 key-value heads, head size 64, bias on Q, K, V
Feed-forward	SwiGLU, inner size 4,864; 87.7% of the parameters of each layer
Normalization and position	RMSNorm ($\epsilon = 10^{-6}$); RoPE with $\theta = 10^6$
Context	32,768 tokens
Vocabulary	151,936 embedding rows; the tokenizer has 151,665 entries
Output layer	Tied to the input embedding
Stored precision	BF16

2.2 Lean environment and the checker

All Lean checks use Lean `v4.10.0-rc1` with mathlib commit `29dcec074de168ac2bf835a77ef68bbe069194c5`. This is the pair that the LeanDojo Benchmark 4 archive records. The checker accepts a candidate only if all of these conditions hold:

1. The target theorem statement is unchanged.
2. The file compiles without an error.
3. The target is absent from the environment before the candidate runs.
4. The proof uses only permitted axioms. `sorry`, `admit`, new axioms, and `native_decide` are rejected as forbidden constructs.

Each run that used Lean checked a true control and a false control before it scored candidates. A verifier infrastructure failure gives a null outcome, which Section 3.4 defines. The checker module is loaded from the run environment; its source is not in this repository. Section 10 audits the scorer of the later Lean-reward trainer, which builds on this checker.

2.3 Compute and accounting

GPU runs used single NVIDIA L40S instances on AWS, as Spot or on-demand leases with an independent termination guard on cpu-angad. CPU work, recounts, and evaluations of saved checkpoints ran on cpu-angad. The approved ceilings were $10 per pilot on 3 October, $64 in total on 4 October, and $95 in total on 6 October. The run ledgers hold conservative cost reservations. They hold no billed amounts, so this report states no billed total.

Each run counts generated tokens and physical Lean calls from its raw records. Evaluation tokens are counted apart from training tokens. No run measured GPU-active time. Wall time includes generation, optimization, evaluation, and Lean, so it is not a GPU time.

2.4 Grading

Natural-language answers in Experiment 9, Experiment 10, Experiment 11, and Experiment 13 were graded by AI reviewers. Reviewers saw blinded responses and references, and a third reviewer

adjudicated disagreements. Planted controls checked consistency. They did not measure accuracy on new answers. No grade has human or Lean validation. Section 3.6 gives the grading contract.

3 Formalization

3.1 Capability decomposition

We decompose mathematical competence into four conditional task families. Table 3 lists them. These are task scores under declared support. They are not direct measurements of separate internal faculties.

Table 3: Capability decomposition used by the evaluations.

Component	Controlled task	Main inference limit
Knowledge access	Recognition, recall, supplied use, and independent use	A prompt cue can reveal a fact or a method
Proof semantics	Execute a local operation with the mathematics supplied	A correct witness can accompany an incorrect explanation
Proof strategy	Choose subgoals and dependencies with facts available	A supplied lemma can reveal the strategy
Integrated solving	Complete a fresh problem without task-specific support	Libraries, search, or repairs can supply missing capability

For divisibility, recall of the existential definition and composition of supplied witnesses are different tasks. Given $b = ak$ and $c = bl$, the witness $w = kl$ satisfies $c = aw$. A correct product witness tests execution. Retrieval of the definition and construction of the argument without support test independent use.

3.2 Mathematical records

Each verified reference derivation has one canonical record

$$E_i = (\Gamma_i,\, g_i,\, \mathcal{K}_i,\, G_i,\, \pi_i,\, R_i). \tag{1}$$

Here Γ_i is the set of assumptions, g_i is the goal, and $\mathcal{K}_i$ is the set of annotated mathematical prerequisites. G_i is the scoped dependency graph, π_i is the sequence of proof operations, and R_i is the renderer and verifier provenance. A theorem can have more than one valid derivation. A grader must not reject a valid alternative because it differs from G_i.

The controlled studies use three record types:

$$\begin{aligned} k = (&\text{id},\ \text{domain},\ \text{statement},\ \text{assumptions},\ \text{prerequisites},\\ &\text{valid_uses},\ \text{invalid_uses},\ \text{source},\ \text{review_status}), \end{aligned} \tag{2}$$

$$\begin{aligned} o = (&\text{id},\ \text{input_state},\ \text{required_conditions},\ \text{action},\\ &\text{output_state},\ \text{justification}), \end{aligned} \tag{3}$$

$$\begin{aligned} e = (&\text{id},\ \text{family_id},\ \text{concept_ids},\ \text{operator_ids},\ \text{assumptions},\\ &\text{goal},\ \text{reference_steps},\ \text{dependencies},\ \text{split},\ \text{provenance}). \end{aligned} \tag{4}$$

A knowledge item k is a fact with its domain and conditions. An operator o is a local transformation with its required conditions. An example e is a task instance with a reference derivation. Each

reference step records its claim, operation, premises, scope, and justification. Every premise of a step must be an assumption, an exposed fact, or an earlier step.

The records keep integer and natural-number domains separate. They also keep zero and sign conditions explicit. For example, modular cancellation belongs to the integers, and its coprimality hypothesis belongs to the rule. A paired invalid use omits that hypothesis and is labeled as an error.

Each record has exactly one admission status: `unreviewed`, `accepted_by_exact_check`, `accepted_by_mathematical_review`, `disputed`, or `rejected`. Only admitted records enter positive training targets. An exact arithmetic check certifies its declared computation. It does not certify a universal statement. Without Lean, a reviewed proof is mathematically reviewed, not formally certified.

3.3 Information views

The model sees a rendered view $x(e, v)$ of an admitted example e, with

$$v = (m_K,\ m_S,\ \rho,\ \tau). \tag{5}$$

Here m_K is the knowledge mask, m_S is the structure mask, ρ is the representation (numerical or symbolic), and τ is the task type. Model-visible fields contain only the rendered task and the declared support. Private fields contain the reference steps, hidden targets, review records, and scoring candidates.

Both masks are discrete intervention labels, not calibrated difficulty values:

- Knowledge axis m_K: full fact supplied, partial fact supplied, recognition alternatives, fact absent.
- Structure axis m_S: local operation supplied, sequence or subgoals supplied, partial sequence, structure absent.

The two axes are not independent in practice. A subgoal can reveal a witness, which is knowledge leakage. A supplied lemma can reveal the decomposition, which is structural support. Each view records exactly which facts and hints it exposes.

With coordinates (knowledge demand, structure demand), the research plan defines three proposed paths: structure first, $(0,0) \to (0,1) \to (1,1)$; knowledge first, $(0,0) \to (1,0) \to (1,1)$; and joint, $(0,0) \to (1,1)$. These paths change both the order and the distribution of support conditions, so they are path comparisons, not order-only comparisons. None has run.

3.4 Learning signals

Verified binary reward. For a task with a frozen formal obligation, let a be the object extracted from the response, such as a witness expression or a proof body. The reward is

$$r(x, a) = \begin{cases} 1 & \text{if Lean accepts the unchanged obligation with } a, \\ 0 & \text{if Lean rejects } a, \text{ or no valid object can be extracted,} \\ \bot & \text{if the verifier infrastructure fails.} \end{cases} \tag{6}$$

A null reward $\bot$ is excluded from updates. It is never converted to 0. Extraction failures receive a separate format status. An accepted witness certifies the submitted object. It does not certify the explanatory prose around it.

RLP prediction reward. Let x be the prompt, r the generated reasoning, and y a hidden target span. Let $\ell_\theta(y \mid \cdot)$ be the mean teacher-forced log probability of y under the policy, and $\ell_{\bar{\theta}}$ the same quantity under an exponential moving average (EMA) teacher. The reward is

$$R_{\mathrm{RLP}}(x, r, y) = \ell_\theta(y \mid x, r) - \ell_{\bar{\theta}}(y \mid x). \tag{7}$$

This objective follows the information-gain reward of RLP [2]. Our task-conditioned adaptation is not a reproduction of the published experiments. The reward scores how much the reasoning raises the likelihood of the target. It does not score whether the reasoning is correct.

Group-relative advantage. For one prompt, the trainer samples G responses $r_1, \ldots, r_G$ with rewards $R_1, \ldots, R_G$. The group-relative advantage is

$$A_j = \frac{R_j - \bar{R}}{s_R + \varepsilon}, \qquad \bar{R} = \frac{1}{G} \sum_{k=1}^{G} R_k, \tag{8}$$

where s_R is the group standard deviation of the rewards and $\varepsilon > 0$ is a small constant [3]. Two consequences follow from Equations (7) and (8).

First, the teacher term cancels. Within a group, $\ell_{\bar{\theta}}(y \mid x)$ is the same for every response, so

$$R_j - \bar{R} = \ell_\theta(y \mid x, r_j) - \frac{1}{G} \sum_{k=1}^{G} \ell_\theta(y \mid x, r_k). \tag{9}$$

The standard deviation s_R is also unchanged by a common shift. Therefore the EMA teacher does not act as a penalty that keeps the policy near its start. The runner used no KL reference penalty in the RLP studies.

Second, a group without reward variation gives no learning signal. If $R_1 = \cdots = R_G$, then $A_j = 0$ for every j. Without a reference penalty, the policy-gradient term for that group is zero. With the binary reward of Equation (6), a group of all failures is such a group. For a binary reward with success probability p per sample, the probability that a group has variation is

$$P_{\mathrm{mix}}(p, G) = 1 - p^G - (1 - p)^G. \tag{10}$$

With $G = 4$, $P_{\mathrm{mix}} = 0.185$ at $p = 0.05$ and 0.875 at $p = 0.5$.

Readiness gate. Before any optimizer update, sample the unchanged policy on the frozen training pool. Let $\mathcal{G}$ be the set of sampled groups. Define the mixed-group fraction

$$\phi_{\mathrm{mix}} = \frac{|\{\, g \in \mathcal{G} : \text{group } g \text{ contains both accepted and rejected responses} \,\}|}{|\mathcal{G}|}. \tag{11}$$

A run proceeds only if $\phi_{\mathrm{mix}} > 0$ across more than one task family. Report the entire pool, including unsolved tasks. Do not select a favorable subset and call it readiness.

3.5 Curriculum formalism

A curriculum experiment has the following parts.

1. A concept graph $\mathcal{C} = (V, D)$. The nodes V are concepts, and an edge $(u, w) \in D$ states that u is a prerequisite of w. Each edge records its provenance: authored, reviewed, annotation-derived, or Lean-checked.

2. A pool of items. Each item has an identifier, its concepts, a split, a task type, and a success check.
3. An ordered list of support levels $s_1 \succ s_2 \succ \cdots \succ s_L$, from most to least support.
4. A presentation multiset $\mathcal{P} = \{ (e,\ v,\ j) \}$, where e is an example, v is a view, and j is a repeat index. A repeat with $j > 0$ is a replay.
5. An arm a, defined by an order function σ_a that maps $\mathcal{P}$ to a sequence.
6. A fading rule. It is either a fixed schedule, or a rule that lowers support only when curriculum validation passes a declared threshold. All arms share the rule.
7. The readiness gate of Section 3.4.
8. Endpoints. Each endpoint states its measure, its denominator, its phase, and its artifact.

An order-only study compares a treatment arm with a control arm. It must satisfy these invariants:

1. Both arms consume the same multiset $\mathcal{P}$, including replay occurrences.
2. Both arms start from the same checkpoint, with the same reward, optimizer, precision, and resource caps.
3. Training, curriculum-validation, and final-evaluation items stay in separate splits. All views and paraphrases of an example family stay in its split.
4. No final-evaluation item enters training, prompts, retrieval, or adaptive sampling.
5. A study that changes the distribution of support conditions is a path comparison, not an order-only comparison.

The experiments used two controls. A per-pass permutation shuffles the order inside each pass over the pool, so membership per pass is identical between arms. A global permutation shuffles the whole sequence $\mathcal{P}$ with a fixed seed, so only the total multiset is identical; a presentation without support can then come before the same example with full support. These controls test different hypotheses, and a study must name which one it uses.

3.6 Evaluation formalism

Knowledge acquisition means improved recall and application of exposed facts on fresh tasks, without those facts supplied at evaluation. Operator learning means improved valid transformations when the required mathematics is supplied. Proof composition means valid combinations of these transformations on unfamiliar dependencies. Delayed retention tests acquisition after intervening updates on other material.

Let C be the set of concept families, and let n_c be the number of unassisted application items for concept c. The primary score of the controlled study is the concept-macro full correctness

$$F = \frac{1}{|C|} \sum_{c \in C} \frac{1}{n_c} \sum_{i=1}^{n_c} \text{full_correct}[c, i]. \tag{12}$$

Unknown grades count as failures in this conservative score, and their frequency is reported. Truncated responses stay in the denominators. The treatment effect of an order study is $\Delta = F_{\text{ordered}} - F_{\text{shuffled}}$, reported with each arm's change from the unchanged checkpoint. The interval

for F uses a paired concept-cluster percentile bootstrap with 10,000 draws and seed 20261006. This interval describes the authored corpus, not variation across training seeds.

The decision rules are fixed in advance. The exploratory progression threshold is a concept-macro gain of 0.10; it is not a power calculation. Delayed retention requires a nonzero success count before the interlude and zero lost successes after it. Checkpoint selection maximizes F on curriculum validation among checkpoints that satisfy retention, with ties to the earliest. Final-test results cannot change the selection, and the RLP reward alone cannot select a checkpoint.

Each graded response returns `fact_correct`, `conditions_correct`, `operations_correct`, `conclusion_correct`, `core_score`, `full_correct`, `first_error_step`, `error_category`, `uncertain`, `reason`, and `evidence`. Graders accept valid alternative methods and conventional notation. They cannot supply a missing central lemma or repair an invalid central argument. A correct conclusion reached through invalid reasoning fails full correctness.

4 Data and capability baselines

4.1 Experiment 1: LeanDojo subset and number-theory pool

Question. Which Lean theorems can serve as training, curriculum-validation, and held-out items, without overlap between splits?

Method. We filtered LeanDojo Benchmark 4 [4] to declared arithmetic and number-theory source files and kept the official `random` split membership. The filter excluded declarations with listed types or ambiguous proof syntax. It removed training declarations whose normalized statements overlap validation or test statements, and training declarations that reference a validation or test theorem. Evaluation exports keep headers, imports, and source locations, and omit proofs. A broader pool used the same curator with logic, sets, functions, relations, integer arithmetic, and more number-theory families.

Result. Table 4 gives the counts. The Nat training split has 447 theorems with tactic traces and 1,312 tactic records. The broader training pool has 1,350 theorems with traces and 5,233 tactic records, and 1,402 annotated references between training theorems.

Table 4: Experiment 1: theorem counts per split.

Export	Training	Curriculum validation	Held-out test
Nat subset: selected before the declaration filter	810	18	15
Nat subset: retained	706	17	15
Broader pool: candidates	2,764	43	38
Broader pool: retained	2,658	43	38

Limits. Text-overlap checks do not establish semantic independence, and the official random split shares source files and theorem families across splits. The LeanDojo authors report that random splits overestimate performance because models can memorize proofs of similar theorems [4]. Lean has not replayed these proofs at the pinned version. Fifteen test theorems give a resolution of 6.7 percentage points per theorem. Model pretraining exposure to mathlib is unknown.

4.2 Experiment 2: Base-model math probe and GSM8K LoRA

Question. What informal mathematics does the base model solve, and does a small supervised adaptation change it?

Method. The probe used `Qwen/Qwen2.5-0.5B` (base, revision `060db649`) through MLX-LM on a laptop, with four-shot prompts and a 512-token cap. It sampled 24 GSM8K and 20 MATH number-theory test questions. Math-Verify scored boxed answers. A LoRA adapter was then trained on 64 separate GSM8K training questions for 32 optimizer updates, with 11,471 teacher-forced tokens.

Result. Table 5 gives the counts. Full-text validation loss on 16 questions fell from 0.701403 to 0.514280, but final-answer accuracy did not improve.

Table 5: Experiment 2: correct final answers. The constant-zero column is a baseline added after the run.

Metric	Base	After LoRA	Constant zero	Questions
GSM8K	9	8	0	24
MATH number theory	3	3	1	20

Limits. These are single-seed sample counts, not benchmark scores. A correct final answer does not establish a correct derivation. The probe used the base model, so it is not the baseline for the Instruct experiments.

4.3 Experiment 3: Instruct proof check

Question. Does the unchanged Instruct model write proofs that Lean accepts?

Method. The local check gave each of 21 formal goals (17 curriculum-validation theorems and 4 elementary goals) one greedy attempt and two sampled attempts, with a 512-token cap and the chat template. It also asked for 8 informal proofs. A second run on an L40S GPU on 3 October sampled the same 21 goals greedily and at temperatures 0.3, 0.6, and 0.9, with four samples per goal at each temperature. Four earlier launches of the GPU run failed during setup, before any generation.

Result. Table 6 gives the counts. Two AI reviewers judged 1 of the 8 informal proofs sound and complete, 2 as a valid core with false explanations, and 5 as invalid or incomplete. Section 9 classifies the 63 failed local attempts.

Table 6: Experiment 3: whole-proof attempts by the unchanged Instruct model on 21 goals.

Run	Attempts	Accepted	At token cap	Generated tokens
Local, recounted	63	0	23	17,326
GPU, greedy	21	0	7	6,569
GPU, temperature 0.3	84	0	18	23,233
GPU, temperature 0.6	84	0	13	19,999
GPU, temperature 0.9	84	0	11	16,563

Limits. The GPU counts come from the TensorBoard export of the streamed run records on cpu-angad. The independent final recount of that run was never done. This diagnostic establishes failure under these prompts and budgets. It does not establish a capability ceiling.

5 Proof RL with a terminal Lean reward

Table 7 describes the curriculum designs of Experiments 4 to 6, 7, 9, and 13 in the terms of Section 3.5. Experiments 4 to 6 used GRPO with the binary reward of Equation (6), TRL 0.24.0, full-parameter training, learning rate 10^{-6}, four samples per group, and no KL penalty. Appendix A lists the settings.

5.1 Experiment 4: Whole-proof GRPO

Question. Does GRPO with a terminal Lean reward improve whole-proof generation, and does source-family order beat random order?

Method. Two arms reloaded the same Instruct checkpoint and saw identical theorem occurrences, in random order or in source-family order. Prompts used training headers without gold traces. The completion cap was 256 tokens. Two GPU launches failed before the completed run. The first failed on a relative checker path. In the second, the curriculum arm aborted during an optimizer save. Three CPU preparation attempts also failed. All failed records are preserved.

Result. Each arm completed 32 optimizer steps and 128 training rollouts. No training proof was accepted. No sampled group had reward variation, and the recorded gradient norms stayed at zero. The unchanged policy and both final policies each solved 0 of 8 diagnostic validation theorems, with 0 of 32 attempts accepted. The worker generated 39,452 tokens.

Interpretation. This is the zero-variance case of Equation (8). The optimizer steps carried no learning signal, so the run says nothing about curriculum order.

5.2 Experiment 5: Prerequisite curriculum GRPO and the syntax retry

Question. Do authored prerequisite tasks with decreasing support give the model enough successes to train on?

Method. We wrote four stages of tasks: logic, arithmetic, divisibility, and induction. Before any update, a readiness check sampled the unchanged policy on 16 training theorems in each support condition, with four samples per theorem. The conditions were lemma statements with definitions and guidance, lemma names only, and no task-specific help. A retry added generic Lean 4 tactic templates to the full-support condition.

Result. Each of the four conditions accepted 0 of 64 sampled proofs. The combined readiness record counts 256 sampled proofs and no accepted proof. No group had reward variation, so the runner skipped GRPO. The original readiness worker generated 30,027 tokens and made 218 physical Lean calls. The failures include whole declarations where a tactic body was requested, obsolete Lean syntax, unknown symbols, and unfinished goals.

Table 7: Curriculum designs, 3 to 6 October 2026. "Not tested" means that the study produced no scored contrast between arms.

Experiment	Order rule	Support schedule	Control	Outcome for the order contrast
4. Whole-proof GRPO	Source-file family rank	None	Per-pass permutation	Not tested. No group had reward variation
5. Prerequisite GRPO	Four authored stages: logic, arithmetic, divisibility, induction	Fixed: lemma statements, then lemma names, then none	Per-pass permutation	Not tested. Readiness accepted 0/64 proofs in each of three support conditions. GRPO skipped
5. Syntax retry	Same as above	Same, with generic Lean 4 syntax templates in full support	Per-pass permutation	Not tested. Readiness accepted 0/64 proofs. GRPO skipped
6. Assisted completion	Prerequisite rank	Fades on validation: missing expression, missing tactic, missing body	Per-pass permutation	Not tested. One theorem qualified, so both arms had the same order
7. RLP continuation	Primitive and proof-structure text first, then textbook contexts	Built into the text	None	No contrast possible
9. Primitive RLP	Four stages with dependencies; replay drawn from earlier stages	Definitions supplied in training only	None	No contrast possible
13. Controlled RLP	Support level: full, partial, recognition, recall, independent	The order is the fading schedule	Global permutation, same multiset	Ordered arm scored. Shuffled arm not scored

5.3 Experiment 6: Assisted proof completion

Question. If the model receives most of a training proof, does it complete the rest, and does that create a learning signal?

Method. Training theorems came with completion forms: a missing expression, a missing final tactic, or a missing body. Evaluation statements had no prefixes. A frozen probe compared four prompt conditions on the same 16 training theorems. A theorem qualified for training when one condition gave both accepted and rejected proofs. Assistance was to fade only when both arms solved enough unassisted validation theorems. CPU replay accepted all 24 reference proofs.

Result. Table 8 gives the counts. Only `PrereqRL.train_twice_divisible` qualified, so both arms trained on one theorem in the same order. Each arm ran three optimizer steps, with two mixed-reward groups that gave nonzero gradient norms. Every unassisted validation round accepted 0 of 16 attempts, so assistance never decreased. The worker generated 48,913 tokens over 440 rollout attempts and made 465 physical Lean calls.

Table 8: Experiment 6: accepted completions, generated continuation tokens, and physical Lean calls.

Phase	Accepted / rollouts	Theorems	Tokens	Lean calls
Frozen, existing prompt	0 / 64	16	7,336	63
Frozen, prefilled code block	0 / 64	16	7,716	64
Frozen, missing expression	2 / 64	16	3,034	64
Frozen, missing tactic	0 / 64	16	7,539	64
Random arm, training	10 / 12	1	449	12
Prerequisite arm, training	10 / 12	1	393	12
Unassisted held-out, baseline	0 / 16	4	2,960	16
Unassisted held-out, random final	0 / 16	4	3,091	16
Unassisted held-out, prerequisite final	0 / 16	4	2,838	16

Interpretation. Assistance created mixed rewards and real gradients. The accepted completions filled in `ih` after a supplied `simpa [twice] using`. The model completed a given proof; it did not find a proof strategy. One qualifying theorem cannot identify an ordering effect.

6 Knowledge acquisition with the RLP reward

Experiments 7, 8, 9, and 13 trained with the RLP reward of Equation (7) through the TRL GRPO trainer, with learning rate 10^{-6}, EMA teacher decay 0.999, and no KL penalty. Experiment 12 audited the training data of Experiment 9. No experiment in this section used a Lean reward.

6.1 Experiment 7: RLP training and primitive-first continuation

Question. Does RLP training on number-theory text improve number-theory answers?

Method. The first stage trained the Instruct model on chapters of an openly licensed algebra textbook (Judson, AATA), with four samples per group, a 128-token completion cap, and a 256-token context. The continuation loaded that policy and its teacher, and trained on project-authored

text about primitives, witnesses, induction, contradiction, and primes, placed before the textbook contexts. Continuation evaluations supplied an answer-format prefix.

Result. The first stage ran 256 optimizer steps over 1,024 rollouts and generated 117,972 training tokens. Its arithmetic evaluation accepted 0 of 32 answers before and after training, and all 32 answers lacked a parseable boxed answer both times. The continuation generated 127,389 training tokens, for 245,361 across both stages. Held-out arithmetic and prime questions scored 11/48 before and after. Prime curriculum validation scored 7/16 before, after, and at every intermediate evaluation. The final policy is saved in S3 under `number_theory_rlp_20261004_continuation/`.

6.2 Experiment 8: Precision audit and FP32 rerun

Question. Did the continuation update the weights at all?

Method. A CPU audit compared every stored parameter of the continuation with its parent, inspected the optimizer state, and ran controlled AdamW updates on the saved weights. A rerun then repeated the continuation from the same parent, teacher, data, seed, and limits, with FP32 parameters and optimizer moments and BF16 computation. Precision was the only intervention.

Result. Only 8,557,971 of 494,032,768 parameter entries changed in the continuation. The saved optimizer moments were BF16, with no FP32 master weights. On identical saved weights with constant nonzero gradients, a BF16 control changed 660 of 65,536 entries, while the FP32 control changed every sampled entry. Both zero-gradient controls left the weights unchanged. In the FP32 rerun, the first update changed sampled weights with FP32 moments. Held-out accuracy stayed at 11/48, and validation went from 7/16 to 6/16. The rerun generated 125,411 training tokens.

Interpretation. The controls show that BF16 rounding discarded small updates in the original setup. The rerun shows that working updates still gave no accuracy gain. The audit does not measure how much the rounding loss contributed to the original null result.

6.3 Experiment 9: Primitive RLP curriculum

Question. Does RLP on authored primitive tasks, presented in dependency order, improve unassisted answers?

Method. The run started from the original Instruct model, not from Experiment 7. It trained on 448 unique authored tasks, from witnesses and substitution through gcd, Bézout coefficients, inverses, primes, and congruences. Replay of earlier tasks expanded them to 596 presentations, one per optimizer step, with eight samples per group and a 512-token response limit. Training prompts supplied definitions. The reward scored a hidden integer target span after the scorer appended a boxed-answer prefix. Evaluation removed the definitions and required the model to write the answer. Curriculum validation had 56 items and the held-out set had 108.

Result. Table 9 gives the held-out counts. Validation scored 16/56 at the start, 25/56 at the first saved checkpoint (`checkpoint-118`), and 13/56 at the end. Between start and end, 28 held-out answers were lost and 7 were gained. Over the same run, the mean reasoned log probability of the target rose from -3.891959 to -2.059282 at 14 fixed positions. The run generated 1,626,779 training tokens over 4,768 training responses.

Table 9: Experiment 9: unassisted held-out answers, 108 items.

Metric	Start	End
Correct boxed integer	42	21
No parseable integer	17	69
Reached the generation limit	15	36
Contains a code fence	0	27

Interpretation. The prediction reward and the correctness score moved in opposite directions. Among 4,768 training responses, 17 incorrect boxed answers received a positive advantage, and a wrong boxed answer outranked every correct one in 2 of 66 mixed groups. So the reward did not consistently prefer wrong answers, but it did not certify correct ones. Training scored a supplied answer prefix, while evaluation required the model to write it. This mismatch is a possible explanation for the decline. No ablation isolated it, and reward choice, optimization, and training length were also not isolated. The early validation peak made `checkpoint-118` a candidate starting point, and the team later selected it. Those validation scores come from repeated measurements on the same items, so they are not held-out evidence.

6.4 Experiment 12: Audit of the primitive RLP corpus

Question. Were the training targets of Experiment 9 mathematically correct, and could shortcuts solve them?

Method. A CPU script recomputed every numeric answer independently of the original preparer. A separate AI-assisted review checked the supplied rules against the source. Constant-answer baselines tested for shortcuts. The audit ran no model, no GPU, and no Lean.

Result. All 612 unique numeric targets were correct. Table 10 gives the review of supplied rules. A constant answer solved every contradiction item: 4 of 4 in validation and 6 of 6 in the held-out set. A constant prime-factor answer scored 4 of 4 in validation and 3 of 6 in the held-out set. Replay changed exposure: witness tasks had 73 presentations and induction tasks had 32, from equal unique counts.

Table 10: Experiment 12: admission of supplied rules for future reuse.

Split	Unique records	Rule review
Training	448	288 reviewed; 160 disputed
Curriculum validation	56	36 reviewed; 20 disputed
Held-out	108	54 reviewed; 30 disputed; 24 admitted by exact check

Interpretation. The disputed rules state remainder, gcd, Bézout, inverse, or cancellation facts without domain or zero conditions. For example, a modular inverse needs $\gcd(a, n) = 1$. Their concrete answers are correct, so the dispute concerns reuse of the rules, not the historical arithmetic. Some task types had shortcuts and cannot distinguish the intended skill.

6.5 Experiment 13: Controlled RLP, ordered arm

Question. Does fading support in a fixed order beat the same presentations in shuffled order?

Method. Both arms continue from `checkpoint-118`. The study has 72 training instances, 18 curriculum-validation instances, and 29 final-test instances. Each instance has five views: full information, partial information, recognition alternatives, domain-cued recall, and an independent problem. Each arm receives the same 432 presentations, including replay. The ordered arm runs from full support to none; the control is a global permutation. Graders scored a five-phase evaluation on the 18 independent-use validation items. Before training, the unchanged checkpoint fully solved 1 of 18 independent-use items and 3 of 18 items with facts and method supplied, for a concept-macro score $F = 0.03125$. On 24 authored controls, each grader made no false acceptance and no false rejection.

Result. The ordered arm completed 432 optimizer steps and generated 2,794,874 training tokens. Its first update changed 68,032 of 68,096 sampled parameters. Table 11 gives its phase results. From the initial to the final phase, 4 items improved in core score, 6 worsened, and 8 stayed the same. No item became fully correct.

Table 11: Experiment 13: ordered arm, 18 validation items per phase. The core score is ordinal partial credit from 0 to 3.

Phase	Mean core score	Fully correct	At the 1,024-token cap
Initial	0.89	0/18	0/18
500,000-token checkpoint	0.89	0/18	2/18
1,000,000-token checkpoint	1.33	0/18	6/18
2,000,000-token checkpoint	1.17	0/18	17/18
Final	0.72	0/18	7/18

Limits. The shuffled arm has responses but no scores, so the order contrast Δ is unmeasured. The 2,000,000-token phase reached the cap on 17 of 18 answers, so its partial-credit score is censored. In this design, order and support level are the same variable: the contrast tests fading support over time against mixed support, not order between concepts. The first second-review file failed evidence alignment and was excluded; a fresh review and blinded adjudication replaced it.

7 Evaluations of saved checkpoints (Experiment 10)

Question. Did the RLP checkpoints of Experiment 9 acquire knowledge that the original model lacks?

Method. Three evaluations compared the original Instruct model, early RLP (`checkpoint-118`), and final RLP on cpu-angad, with matched FP32 inference and greedy decoding. No training occurred. All grades are blinded AI judgments with adjudication.

7.1 Content probe

Table 12 gives the counts. A constant option label scores 6 of 24 on recognition, and a constant application answer of one scores 4 of 12. Two reviewers agreed on 71 of 72 responses. Final RLP

reached the response limit on 12 of 12 recall answers and 11 of 12 application answers. Graders marked 7 of its 12 application answers unassessable, so its zero applications do not show erased knowledge.

Table 12: Experiment 10, content probe: correct responses.

Task	Original	Early RLP	Final RLP
Concept recognition, repeated option orders	19/24	19/24	17/24
Definition and theorem recall	3/12	4/12	3/12
Elementary application, content review	3/12	4/12	0/12
Elementary application, strict integer parsing	3/12	3/12	0/12

7.2 Acquisition, proof semantics, and proof strategy

Table 13 gives complete core answers under a revised rubric, with a 1,024-token response limit. The original model completed 4 of 24 acquisition tasks, and each RLP checkpoint completed 2 of 24. Final RLP generated 63,735 tokens and reached the cap on 26 of 80 main responses. The original generated 40,123 tokens with 2 of 80 at the cap, and early RLP generated 32,975 tokens with 0 of 80. The two independent reviews disagreed on 44 of 321 response views, including the notation check. The rubric was revised after the original responses and strict grades were inspected, so this is post hoc exploratory regrading.

Table 13: Experiment 10, decomposition evaluation: complete core answers.

Task family	Original	Early RLP	Final RLP
Definition and theorem recall	1/12	0/12	1/12
Explained application	3/12	2/12	1/12
Proof-step checking	2/12	0/12	1/12
Complete proof, unaided	2/12	0/12	1/12
Same proof, supplied facts	4/12	0/12	3/12
Same proof, facts and plan	3/12	0/12	2/12
ProcessBench selected subset	0/4	0/4	0/4
ProofNet-Verified selected subset	0/4	0/4	0/4

7.3 Knowledge inventory and Lean witness controls

Table 14 compares the original model with early RLP. Early RLP doubled explained recognition, from 7 of 32 to 14 of 32 complete core answers, but did not improve supplied or independent use. The original reached the cap on 2 of 80 responses and early RLP on 0 of 80. Recognition repeats the same questions with rotated options, so its presentations are not independent concepts.

A Lean adapter for submitted witnesses passed 16 of 16 controls: it accepted every correct witness and rejected every incorrect one. All 4 direct early-model responses failed the expression-only parser. The parser failed on the answer format; one response still contained a valid boxed witness.

Interpretation. Early RLP improved recognition only. It did not improve supplied or independent use. Final RLP lost complete answers, and truncation confounds that loss. We wrote these probes after analyzing earlier failures, so they serve as development checks and cannot serve as fresh held-out tests.

Table 14: Experiment 10, knowledge inventory: complete core answers and fully correct answers.

	Complete core		Fully correct	
Task	Original	Early RLP	Original	Early RLP
Recall	2/16	1/16	0/16	0/16
Explained recognition	7/32	14/32	4/32	8/32
Supplied use	4/16	4/16	1/16	2/16
Independent use	3/16	3/16	2/16	1/16

8 RL with a Lean-verified local reward (Experiment 11)

Question. Does a Lean-verified reward teach the model a small proof step when the mathematics is supplied?

Method. Each task supplied equalities and asked for a witness. For example, given $b = ak$ and $c = bl$, the model must submit kl so that $c = a(kl)$. Lean checked the witness in an immutable obligation, and the reward followed Equation (6). GRPO ran 64 optimizer steps from `checkpoint-118`, with eight samples per group, a 1,024-token cap, FP32 optimizer moments, and a KL coefficient of 0.02. Checkpoint selection used the strict validation score with earliest ties, and selected step 64. Lean gave no feedback during generation.

Result. Table 15 gives the counts. The GPU run generated 135,822 tokens and made 346 physical Lean calls. With the CPU development samples, the totals are 145,873 tokens and 408 Lean calls.

Table 15: Experiment 11: unchanged `checkpoint-118` against the selected checkpoint.

Endpoint	Unchanged	Selected
Validation: strict extraction and Lean acceptance	0/8	6/8
Validation: correct witness, content review	0/8	7/8
Validation: no substantive mathematical error	0/8	2/8
Held-out composition: strict extraction and Lean acceptance	0/4	2/4
Held-out composition: correct witness, content review	1/4	2/4
Held-out composition: no substantive mathematical error	0/4	0/4

Interpretation. The content review shows that the gain includes corrected mathematics as well as formatting. Lean checked only the witness. Several valid witnesses came with explanations that divide by a quantity that can be zero. The trained model still dropped factors on harder compositions. With 8 validation and 4 held-out items, the evidence supports a narrow result about executing one supplied operation. It is too small for a claim about reliability. The content review saw checkpoint labels, so it is not blinded.

9 Post hoc analyses, 8 October 2026

9.1 Why the Instruct proof attempts failed

Question. Did the 63 local attempts of Experiment 3 fail on the mathematics or on the Lean language?

Method. `scripts/audit/classify_proof_failures.py` reads the saved Lean output of each attempt and assigns one exclusive class. It runs no model and no Lean.

Result. Table 16 gives the classes. No attempt reached the point where Lean checks the mathematics. Of the 60 attempts that Lean checked, 44 placed `import` lines inside the proof and 23 contained stray Markdown backticks. Of all 63, 34 wrote a new `theorem`, `lemma`, or `example` declaration when only a proof body was requested. A pattern match finds Lean 3 syntax, such as `begin ... end` or `import data.nat`, in 33 of 63. We checked 6 of those matches by hand, and all 6 were Lean 3. Lean reported no unknown-identifier error, because parsing failed before elaboration, so invented lemma names were never checked.

Table 16: Classes of the 63 failed local attempts of Experiment 3.

Class	Attempts
Parse or file-structure error	57
Empty proof (the response was only `by`)	3
Forbidden construct (`sorry`)	3
Well-formed Lean that failed only on the mathematics	0

Interpretation. The first barrier is the Lean 4 language and the file format, before any mathematics. A prefilled code block did not remove it: in Experiment 6, that condition accepted 0 of 64. The analysis covers one prompt, the unchanged model, and 21 goals. The 256 attempts of Experiment 5 are on cpu-angad and were not reclassified.

9.2 Material for a proof-state curriculum

Question. How many starting points for a curriculum that starts near the end of a known proof do the training proofs contain?

Method. `scripts/audit/curriculum_inventory.py` reads the traced training proofs. It counts a proof as linear when each tactic's resulting state equals the next tactic's starting state and the last state has no goals. A linear proof with n tactics gives $n - 1$ starting points with at least one reference tactic supplied. The script reads training proofs only, because evaluation records carry no proofs.

Result. Table 17 gives the counts. In the broader pool, 85% of the linear proofs have one or two tactics, and only 24 have five or more. The 490 nested tactic proofs have a median of five tactics. No validation or test theorem appears as a premise in a training proof.

Limits. The linearity test compares state strings, and Lean has not replayed the proofs. Without cutting the nested proofs, most theorems allow at most two curriculum levels. The references are annotations, not kernel-checked dependencies.

10 Audit of the Lean reward, 9 and 10 October 2026

Question. Which inputs earn the Lean reward without a proof of a statement worth proving?

Table 17: Proof structure of the training exports of Experiment 1.

Count	Nat subset	Broader pool
Training theorems	706	2,658
Term-mode proofs, no tactics	259	1,308
Linear tactic proofs	295	860
of which one tactic	200	485
of which five or more tactics	5	24
Nested tactic proofs	152	490
Starting points with a supplied prefix	150	600
References between training theorems	344	1,402

Scope. The audit covers the scorer of the Lean GRPO trainer (`scripts/train/run_lean_grpo.py`). The probe and the pass-rate map share this scorer. At this audit snapshot, the trainer had not run on a GPU. The attack inputs in Sections 10.2 to 10.6 were constructed audit cases, rather than observed policy exploitation. Later GRPO training is recorded separately in Section 1.2. Those entries came from a script that we wrote or from a data audit. Entries R1 and R4 come from earlier experiments. A *scorer exploit* is a completion that the scorer accepts although it is not a proof of the stated target. Experiments 3, 4, and 5 accepted no proof, so a false accept could not have raised their counts. This section does not re-audit the checker runs of Experiments 1 to 13. The same inventory, with the attack inputs, is in `docs/REWARD_HACKING.md`.

Table 18: Inventory of inputs that earn the Lean reward without a proof of a statement worth proving. "Exploit" says whether the input needs a scorer exploit. SFT is supervised fine-tuning on Lean proofs.

ID	What earns reward	Exploit	Status
S1	Forged axiom report from `trace` or `run_tac`	Yes	Fixed
S2	Replaced `#print axioms` elaborator	Yes	Fixed
S3	Command after the proof	Yes	Fixed
S4	Namespace shadowing of the target name	Yes	Fixed
P1	Valid proof followed by an extra declaration	Yes	Fixed
P2	Library search with `exact?` or `apply?`	No	Fixed by the forbidden-construct filter
P3	Library search with `observe`	No	Open
T1	One tactic on a statement that one tactic proves	No	Excluded from the RL pool. 99 statements remain in SFT problems
T2	Valid proof of a vacuous statement	No	Excluded from training. Detection is incomplete
T3	Valid proof of a mis-formalized statement	No	Partly filtered
L1	Recall of proofs from evaluation files	No	Fixed in task bank v3
L2	Recall of near-duplicate training proofs	No	Capped, not removed
L3	Copy of the proof of a near-identical training statement	No	Measured as a baseline
L4	Proof that cites an evaluation theorem by name	No	Rule stated, not tested

ID	What earns reward	Exploit	Status
L5	Dev set that shares files with SFT data	No	Replaced by a dev slice carved from train
W1	Goal closed through a sorried lemma in REPL tactic mode	Weaker check	Not used as the reward
W2	Five exploits in the old subprocess check	Weaker check	Open
W3	Trailing declaration in the batch-file check	Weaker check	Open for external proofs
R1	Correct witness with a wrong explanation	No	Observed in Experiment 11
R2	Constant answer	No	Measured as a baseline
R3	No-op tactics under a per-step reward	No	Control stated, not implemented
R4	Rising RLP reward with falling accuracy	No	Observed in Experiment 9. Cause untested
N1	Reward noise from batch-file neighbours	Not a gain	Fixed in the probe
N2	Reward that depends on machine load	Not a gain	Open
N3	Reward that favors short outputs	Not a gain	Open

10.1 What the trainer pays

By Equation (6), the trainer gives reward 1 when the scorer accepts a completion, and 0 otherwise. The function `verify_group` returns the verdict. The trainer, the probe (`scripts/eval/probe_nt_pool.py`), and the pass-rate map (`scripts/eval/pass_rate_map.py`) call it. It accepts a completion only if all of these hold:

1. The completion has no match of the pattern `FORBIDDEN`. A match gives a rejection without a Lean call.
2. The completion is not empty, and every line is indented.
3. The fixed prefix, the supplied steps, and the completion parse as one command.
4. Lean reports no error within 20 seconds. A timeout or a REPL crash is a rejection.
5. The target guard `TARGET_ABSENT` of the checker passes.
6. One info message sits at the line of the checker's `#print axioms` command and names the target. It lists only `propext`, `Classical.choice`, and `Quot.sound`.

The REPL is the Lean read-eval-print loop (leanprover-community/repl, tag `v4.10.0-rc1`). It checks each completion as one command in a loaded environment. The pattern `FORBIDDEN` is in `scripts/eval/probe_start_depth.py`. It matches the words `sorry`, `admit`, `axiom`, `unsafe`, `native_decide`, `set_option`, `run_tac`, `run_conv`, `by_elab`, `run_cmd`, `run_elab`, and `run_meta`. It matches the symbol `#`. It also matches `exact?`, `apply?`, `rw?`, `rw_search?`, `hint`, and `library_search`.

Table 19 labels each reward component.

10.2 Inputs that the scorer accepted without a proof

Method. `scripts/eval/check_checker_attacks.py` holds 29 attacks and 2 known-good

Table 19: Reward components and their labels.

Reward	Label	Status
Binary `accepted` from `verify_group`	Lean-verified	Implemented. Not run on a GPU
Rewards shuffled within each group	Not Lean-verified for the completion that receives it. Control arm only	Implemented. Not run on a GPU
Per-step reward	Heuristic. Lean computes it, but it measures progress, not proof	Designed. Not implemented
RLP reward, Equation (7)	Heuristic. No Lean check	Run in Experiments 7, 8, 9, and 13
Witness reward, Equation (6)	Lean-verified for the witness only	Run in Experiment 11

controls. An attack is a test input that tries one exploit. Each case runs on 3 train statements, so one run makes 93 checks. The script scores each continuation with `verify_group`, as the trainer does.

Result. The scorer before the fix (code folder `train_code_v5`) accepted 27 of 87 attack checks. That is 9 attacks, each accepted on all 3 statements. Eight of the 9 prove nothing. Each closes the goal with `exact sorryAx _ false`, and a true axiom report would list `sorryAx`. The ninth is a valid proof followed by an extra declaration. The pattern `FORBIDDEN` matches the word `sorry` but not `sorryAx`. The axiom report was therefore the only guard against `sorryAx`, and the attacks of Table 20 defeated it.

Table 20: The 9 attacks that the scorer accepted before the fix. Each ran on 3 statements, from `trainer_checker_v1/attacks_prefix2`.

ID	Attack	Why the scorer accepted it
S1	`trace_fake_no_axioms`	The proof prints a forged "does not depend on any axioms" line with `trace`. The parser accepted that text anywhere in the output
S1	`trace_fake_axiom_list`	The proof prints a forged axiom list. The parser took the first match, and the forged line came before the real report
S1	`run_tac_fake_report`	The same forged line, printed by a metaprogram
S2	`run_tac_override_print_axioms`	A metaprogram adds a new elaborator for the `#print axioms` command. The checker's own command then prints the forged report at the right line, with the right name
S2	`by_elab_override_print_axioms`	The same elaborator, added through the term `by_elab`
S3	`command_elab_override`	A Lean command can start at any column. An indented `@[command_elab]` definition after the proof was a new top-level command. The precheck rejected only unindented lines
S3	`trailing_run_cmd_fake_report`	An indented `run_cmd` after the proof prints the forged report
S4	`namespace_shadow`	After `namespace Shadow`, the checker's `#print axioms` named `Shadow.<name>`, whose report was clean. The scorer did not check which constant the report named
P1	`trailing_theorem`	A valid proof followed by an extra `theorem`. This is a policy violation, not a false proof

Fix. Three rules in the shared scorer close these attacks.

1. The scorer reads axioms only from the one info message at the line of the `#print axioms` command. The message must name the target (`axiom_report` in `scripts/eval/bench_lean_verifier.py`).
2. A `run_cmd` in the same REPL command parses the declaration with Lean's own parser. It fails with `EXTRA_COMMAND` if a second command follows (`one_command_check`). It runs before the candidate text, so the candidate cannot change it.
3. `FORBIDDEN` now matches `run_tac`, `run_conv`, `by_elab`, `run_cmd`, `run_elab`, and `run_meta`. Rules 1 and 2 cannot stop code that changes the `#print axioms` elaborator.

We switched off one rule at a time on the statement `lean_workbook_1155`. Table 21 shows that each rule is needed. After the fix, 0 of 87 attack checks were accepted and 6 of 6 controls were accepted (`trainer_checker_v1/attacks_v9`, exit code 0). The controls ran after a REPL crash and a REPL timeout in the same call.

Table 21: Attacks accepted again when one rule is switched off, on `lean_workbook_1155`.

Rule switched off	Attacks accepted again
Rule 1: axiom report by position and name	`trace_fake_no_axioms`, `trace_fake_axiom_list`
Rule 2: one-command check	`command_elab_override`, `trailing_theorem`
Rule 3: `FORBIDDEN` additions	`run_tac_override_print_axioms`, `by_elab_override_print_axioms`

Earlier accepted proofs. The fixed scorer re-scored the proofs that earlier runs accepted (Table 22). A text scan also looked for what each exploit needs: metaprogram words, `logInfo`, `trace`, `command_elab`, and a line that starts with a command keyword. The scan covered 10,869 accepted external proofs. It found no metaprogram word and no such line. Five proofs matched the word "depend" inside Goedel comments. The 7 SFT proofs that time out at 20 seconds also time out on the scorer before the fix, and the fixed scorer accepts all 7 at a 120-second limit. No proof accepted in these runs used an exploit.

Table 22: Earlier accepted proofs, re-scored with the fixed scorer.

Source	Accepted before	Accepted by the fixed scorer
`train200_v3` shards (single tactic)	24	24
`train_v2_single_tactic`	393	393
`val_v2_single_tactic`	55	55
`nt_sft_v2` train and validation rows	1,689	1,682, and 7 timeouts at 20 s

Limits. The re-scoring report does not list the LeanDojo 43-theorem evaluations or the step-level search runs. The attack list is finite, so a count of zero accepted attacks does not show that the scorer is sound. The scorer depends on `checker.py`, which is outside this repository.

10.3 Proofs that the project does not want paid

Library search (P2). The tactic `exact?` passed the acceptance check: Lean exited with code 0, printed "Try this: exact `Nat.dvd_trans` h_1 h_2", and listed only allowed axioms. The single-tactic baseline on the 43 LeanDojo validation theorems was 13/43 with `exact?` and 9/43 without it. The baseline ran 430 attempts, 10 tactics on 43 theorems, and `exact?` alone solved 8 theorems. `FORBIDDEN` now blocks `exact?`, `apply?`, `rw?`, `rw_search?`, `hint`, and `library_search`. Use 9/43 as the baseline.

The tactic `observe` (P3). The Mathlib tactic `observe` is equivalent to `have hp : p := by exact?`. `FORBIDDEN` does not match it. This is not a soundness exploit, because Lean still checks the proof term. It matters only if the policy must name the lemma itself. No attack was run.

10.4 Valid proofs of statements without information

These entries need no scorer exploit. Lean accepts the proof, and the proof is correct. The reward measures the statement, not the policy.

Single-tactic statements (T1). A *single-tactic statement* is a statement that one tactic from a fixed list of nine proves. The list is `aesop`, `tauto`, `simp_all`, `omega`, `norm_num`, `simp`, `rfl`, `linarith`, and `decide`. Of 3,450 train statements, 161 (4.7%) are single-tactic statements. 393 of 31,050 candidates were accepted. By tactic, the accepted train candidates are `aesop` 97, `tauto` 73, `simp_all` 71, `omega` 60, `norm_num` 31, `simp` 30, `rfl` 14, `linarith` 9, and `decide` 8. Of 420 validation statements, 28 (6.7%) are single-tactic statements, with 55 of 3,780 candidates accepted. On 200 train statements, the SFT model solved 0 (0 of 1,600 samples), and the nine single tactics solved 8 (24 of 1,800 candidates). On the LeanDojo validation set, the SFT model solved 3 of 43 theorems at 8 samples, and all 3 lie inside the 9 theorems that single tactics solve.

A policy that learns to emit `omega` or `simp_all` first would gain reward on these statements with no proof skill. At $n = 420$, an exact two-sided McNemar test at $\alpha = 0.05$ needs 6 newly solved statements and 0 losses. The 28 single-tactic validation statements exceed that floor. The primary metric of the midsemester study is therefore the 392 validation statements that no single tactic proves, reported next to the full 420. The RL pool holds 0 single-tactic statements. One drawback remains: the SFT problems still hold 99 of them, so the SFT model trains on their proofs.

Vacuous statements (T2). A *vacuous statement* has hypotheses that prove `False`. Any conclusion follows, so the proof is trivial. `scripts/eval/false_hypothesis_sweep.py` tries `exfalso` followed by one of 8 closing tactics on all 4,306 pool statements, which makes 34,448 candidates. A statement is vacuous when one candidate is accepted. Two synthetic controls checked the sweep. With $n \in \mathbb{N}$, the statement with hypothesis $n < 0$ and goal $n = 5$ was accepted by 8 of 8 tactics, and the statement with hypothesis $n < 3$ and goal $n < 5$ was rejected by 8 of 8. Table 23 gives the counts. All 17 vacuous train statements have proofs in `nt_sft_v2`, and all 3 vacuous validation statements lie among the 28 single-tactic validation statements. Table 24 gives five hits that we checked by hand.

The sweep has two limits. Detection is incomplete: 16 statements had a timeout or a REPL crash and no accepted candidate, and they stay in the data. Also, `exfalso` does not introduce premises inside the goal, so the sweep tests only the hypotheses before the colon.

Table 23: Statements of `nt_pool_v2` without information, by split. "–" means not measured. Sources: `nt_probe/`, `nt_false_hyp_v1/`, and `nt_pool_v2/summary.json`.

Count	Train	Validation	Test
Statements	3,450	420	436
Single-tactic statements	161	28	–
Vacuous statements	17	3	3
Undetermined (timeout or crash, no accepted candidate)	10	1	5
Dropped by the real-number filter	68	9	9

Table 24: Five vacuous statements, checked by hand.

Statement	Split	Why the hypotheses are false
`lean_workbook_plus_14943`	train	$n = 5k + 1$ and $n = 5k - 1$ give $1 = -1$
`lean_workbook_38542`	train	In $\mathbb{N}$, $3n - m + 5 \geq 5$, so it cannot equal 1
`lean_workbook_plus_64022`	train	The recurrence at $n = 0$ gives $a_1 = a_0$, so $1 = 2$
`lean_workbook_27418`	validation	$(n, p) = 1$ is an equation in $\mathbb{N} \times \mathbb{N}$, so $p = 1$, which contradicts `p.Prime`
`lean_workbook_50556`	test	In $\mathbb{N}$, $3/2 = 1$, so `h3p` says $p < n$, which contradicts `hp : n < p`

Mis-formalized statements (T3). A *mis-formalized statement* is a Lean statement that does not say what its source problem says. The statement `lean_workbook_plus_38640`, $10^n \mid (5+\sqrt{35})^{2n-1}$, is divisibility in a field, so it is trivially true. The pool filter checked only for $\mathbb{R}$ and $\mathbb{C}$. It now also matches the square-root sign, the symbol π, `Real.`, and `Complex.`, and it dropped 86 statements (Table 23). Subtraction and division in $\mathbb{N}$ cause several vacuous statements, as Table 24 shows. The filter does not catch three statements that the coordinator named: `lean_workbook_48558` (a definition of `lcm`), `lean_workbook_plus_52344` (a set tautology), and `lean_workbook_plus_75144` (a closed count with no free variable). Of the 8 train statements that single tactics solved in the 200-statement run, at least 5 were trivial or mis-formalized.

10.5 Valid proofs that the policy can recall

These entries need no scorer exploit. Reward rises because the policy recalls content, not because it proves something new. *Contamination* is evaluation content in the training data, the prompts, or the RL tasks. The *token Jaccard similarity* of two statements a and b is $J(a, b) = |T_a \cap T_b|/|T_a \cup T_b|$, where T is the set of tokens of the statement text without its name and proof. A *near-duplicate* pair has J at or above a stated cap.

Task bank overlap (L1). 826 of the 1,007 accepted tasks of task bank v1 lie in the 26 LeanDojo evaluation files. 1,137 of its 1,350 records lie there. Task bank v3 had 0 of 4,632 records in evaluation files at the time of the check. RL tasks now come only from files outside the evaluation files.

Near-duplicates (L2). The split key hashes statements after it renames variables. It misses hypothesis-name and parenthesis variants. The token scan found 68 train statements at $J = 1.00$ to a validation or test statement. 13 of them are SFT problems. The key had missed all 68. Splits v3 exclude train statements at $J \geq 0.85$ to a validation or test statement, and at $J \geq 0.7$ to a miniF2F statement. They exclude 165 near-duplicates of a test statement, 127 of a validation statement, 34 of

a miniF2F statement, 17 vacuous statements, and 16 miniF2F-check names. One statement can have several reasons. A cap leaves close statements below it: in splits v2, 192 of 420 validation statements have a train neighbour between 0.70 and 0.85. Report validation results by nearest-train-similarity band. The miniF2F statement `aime_1994_p3` equals `lean_workbook_plus_44384` up to hypothesis names, and that statement and `lean_workbook_plus_59411` are miniF2F near-duplicates that `nt_sft_v2` contains.

Copying the nearest training proof (L3). `scripts/eval/nearest_proof_baseline.py` takes no model. It pastes the Lean-accepted proofs of the 8 nearest train statements under each validation header. Table 25 gives the counts. 5 of the 22 non-trivial solves copy a train statement at $J = 1.0$. All 5 neighbours have proofs in `nt_sft_v2`. A model trained on `nt_sft_v2` therefore saw proofs that copy-solve 5 validation statements, and its validation numbers need this caveat. The baseline pastes bodies verbatim. Renaming hypotheses would likely solve more, and we did not test it. Report this baseline next to every arm.

Table 25: Validation statements solved by copying a train proof, with no model (`trainer_checker_v1/nearest_proof_v7` and `nearest_proof_trainable_v8`). The non-trivial set excludes the 28 single-tactic statements.

Library of train proofs	All 420	Non-trivial 392
Full train split	37	22
Trainable (285 statements dropped)	26	12

Citations of evaluation theorems (L4). The external proof check found 43 citations of evaluation theorem names in accepted proofs, over 13 distinct names. The detector misses dot notation. A proof that cites a held-out theorem passes by lookup. The rule is to drop near-duplicate problems and citing proofs before use.

Dev sets that share files (L5). For the LeanDojo dev sets, 147 of 148 theorems (v1) and 263 of 266 theorems (v2) share a file with the SFT training set. Decisions on those sets measure recall. The number-theory study uses a 300-statement dev slice carved from train that is in neither the SFT data nor the RL pool.

10.6 Weaker checks that accept a non-proof

REPL tactic mode (W1). The tactic mode of the REPL has no axiom check. In the step-level search over LeanDojo theorems, 9 closed paths for `Set.union_diff_left` in `search_v1_sftfinal` were rejected on re-check, because the proofs use a lemma whose proof had been replaced by `sorry`. The fixed-list search `search_v3_fixed`, which uses no model, closed 29 paths for the same theorem, and all were rejected. The same mode rejects 6 of 20 correct reference proofs. The GRPO reward uses the environment mode of the REPL, which checks axioms. It agrees with the subprocess check on 160 of 160 proofs. Every found proof needs an axiom check on re-check.

Old subprocess check (W2). The function `check` in `scripts/eval/probe_start_depth.py` reads axioms from the whole Lean output and has no one-command check. By code reading, it keeps five exploits: `trace_fake_no_axioms`, `trace_fake_axiom_list`, `command_elab_override`, `trailing_theorem`, and `namespace_shadow`. This was not tested. Three scripts call it: `lean_e`

`val_statements.py`, `single_tactic_baseline.py`, and `search_tactics.py`. The 9/43 baseline and the 3/43 SFT result come from them, and the re-scoring of Table 22 does not list those runs. Do not use these scripts to score model output until they call `verify_group`.

Batch-file check (W3). The function `check_batch` in `scripts/data/verify_external_proofs.py` reads each axiom report by line, so the forged reports of S1 and S2 cannot work there. It has no one-command check. It scores external proofs only, not model output.

10.7 Reward that covers part of the output, and reward noise

Witness-only reward (R1). In Experiment 11, Lean checked one witness. Correct witnesses on validation rose from 0/8 to 6/8, and error-free answers were 2/8 on validation and 0/4 held-out. The explanations around the witness often stayed wrong.

Constant answers (R2). The corpus audit of Experiment 12 found that a constant answer solved every contradiction item. On the answer-task validation set, a constant answer scores 14/309: `gcd` 5/45, `pow_mod` 6/45, `mod_inverse` 2/45, and `witness` 1/45. A random answer scores 0/309.

No-op tactics (R3). A per-step reward would credit the fraction of generated tactics that elaborate before the first error. Tactics that elaborate and change nothing, for example `skip` or `have h := h`, would earn it. A subgoal reduction is not a proof. The planned control counts a tactic only if the goal state changes, keeps the per-step weight below the proof weight, and reads the ten highest-reward samples each round. None of this is implemented.

RLP reward and accuracy (R4). In Experiment 9, the RLP reward rose while held-out accuracy fell from 42/108 to 21/108. The reward is heuristic and has no Lean check. A train and evaluation mismatch is a possible cause, and no test isolated it.

Batch-file neighbours (N1). A candidate that ended after `by` or a bullet made Lean report the parse error in the next candidate's span. An unterminated block comment swallowed the rest of its file. In one test, 15 of 40 known-good proofs were rejected. The probe now scores with the REPL path. Re-scoring 1,229 old probe verdicts gave 0 flips.

Timeouts (N2). 7 of 1,689 SFT proofs need more than the 20-second limit under the load of cpu-angad. At a 120-second limit they take 24.6 to 84.1 seconds each. The single-tactic checks gave 23 timeouts and 11 REPL crashes in 31,050. The scorer gives reward 0 to a timeout and to a crash, whereas Equation (6) reserves $\perp$ for verifier failure. A policy that avoids slow tactics would gain reward. This is an inference, not a measurement.

Length cap (N3). 253 of 1,600 first errors (15.8%) of the SFT model were the 256-token cap. A binary reward pays outputs that stop early.

10.8 Checks before an RL gain counts, and open items

Checks. Run these before any RL gain is reported:

1. Split the solved set of every arm into single-tactic, vacuous, near-duplicate, and other statements. Report the rate on the other statements as the headline.

2. Report the nearest-proof baseline of Table 25 next to every arm.
3. Run `scripts/eval/check_checker_attacks.py` on the final scorer. Require 0 accepted attacks and all controls accepted.
4. Run `scripts/eval/rescore_accepted.py` on every accepted proof of the final run, and read the scan hits.
5. Read the highest-reward samples of each round.
6. Run the shuffled-reward arm. It keeps the reward rate and the share of mixed groups, and it breaks the link between a completion and its reward. A *mixed group* holds both a reward of 0 and a reward of 1.
7. Report the cap rate, the mean length, and the timeout count per arm.

Open items. These come from the trainer-and-checker report. As of 10 October, the decision log of `docs/EXPERIMENTS.md` has no line on items 1 to 4 and 6.

1. Accept the three scorer rules as the reward check for every arm.
2. Retire the old subprocess check, or move its three users to `verify_group`.
3. Decide whether `observe` joins `FORBIDDEN`.
4. Keep the 20-second timeout, or raise it and pay more Lean time per step.
5. Use the new SFT data for the midsemester arms, or report the caveat of Section 10.5.
6. Add a dated line to the decision log of `docs/EXPERIMENTS.md` for the scorer change and for the removal of the Bernoulli random-reward arm.

Remaining risks. The REPL has no memory cap of its own, so a large computation can allocate memory before the 20-second timeout. Every job needs a systemd `MemoryMax`. Two older copies of `FORBIDDEN` remain, in `scripts/data/replay_training_proofs.py` and `scripts/data/build_sft_data.py`. They define weaker patterns and filter data. They do not score reward.

11 What the record shows

The cold start blocked every terminal-reward study. Equation (8) gives zero advantage to a group of all failures, and Equation (10) shows how fast such groups dominate as the success rate falls. Experiments 4 and 5 had no successes. Experiment 6 created successes only with most of the proof supplied. Section 9 shows that the unchanged model did not produce well-formed Lean 4, so the barrier came before the mathematics.

A verified reward works on a task the model can sometimes solve. Experiment 11 narrowed the task to one supplied local operation. Groups then had variation, and validation accuracy rose from 0/8 to 6/8. The explanations around correct witnesses often stayed wrong, so the reward certified the submitted object only.

The prediction reward does not track correctness. In Experiment 9 the reasoned target likelihood rose while held-out accuracy halved. In the preparation of Experiment 13, a deliberately false reasoning prefix raised the target likelihood in both inspected examples. RLP gave no accuracy gain in Experiments 7, 8, 9, or 13.

No curriculum comparison has a result. Three studies had no reward variation, one selected a single task, two had no control arm, and one has an unscored control. Every prerequisite edge so far was authored by the team or derived from annotations. No experiment tested whether a prerequisite helps a later concept.

Grading without a checker is noisy. AI reviewers disagreed on 44 of 321 views in Experiment 10, and the primitive RLP corpus contained shortcut tasks (Experiment 12). This motivates Lean-checked endpoints where they exist.

A Lean-checked reward is not a proof of skill. Section 10 shows two limits. Before a fix on 10 October, the scorer of the Lean GRPO trainer accepted 9 kinds of input that are not proofs of the target. After the fix, it accepted none of the 29 attacks. Separately, a single tactic from a fixed list proves 28 of 420 validation statements, and copying the nearest training proof solves 22 of the 392 others. A rise in the accepted rate needs a split by statement type before it counts as proof skill.

11.1 Threats to validity

- Samples are small. Validation sets have 8 to 56 items, the Nat test split has 15 theorems, and the false-prefix result rests on 2 examples.
- Each run used one training seed, so no result has a seed-to-seed interval.
- Several evaluation sets were reused across experiments and became development material.
- Pretraining exposure of the model to mathlib and to the authored concepts is unknown.
- No run measured GPU-active time, so cost comparisons use tokens and Lean calls only.
- The GPU sampling counts of Experiment 3 come from an exported monitoring stream. Nobody recounted them independently.
- The counts of Section 10 come from run reports on cpu-angad. Nobody re-read them from the raw files.
- The attack list of Section 10 holds 29 inputs. A count of zero accepted attacks does not show that the scorer is sound.

12 Proposed next study (not run)

This section states a plan. Nothing in it has run, and its predictions are declared before any data.

Question. Under a fixed Lean-call budget, which gives more held-out Lean proofs for a 0.5B model: a curriculum that keeps the success rate near one half, or a reward that credits individual tactics?

Stages. The plan has four stages:

1. Supervised training on Lean 4 proofs at the pinned version, so that the model writes well-formed Lean. The parse rate and the elaboration rate on held-out statements decide when this stage ends.

2. Expert iteration: sample proofs on fixed statements, keep the proofs that Lean accepts, and fine-tune on them [5]. RLP on the same corpus enters only if it beats supervised training alone at equal compute.
3. GRPO with the binary reward, once the readiness gate passes.
4. A comparison of GRPO with expert iteration at the same number of Lean calls. On informal mathematics, Havrilla et al. report that "all algorithms perform comparably, with Expert Iteration performing best in most cases" [6].

Curriculum. A reverse curriculum gives the model a reference proof prefix and asks for the rest, then moves the starting point back toward the theorem statement [7]. The arms are: no curriculum; a staged schedule; all depths mixed; and an adaptive rule that keeps each theorem near a success rate of one half. In the R3 study, the mixed schedule beat the staged one [7]. The adaptive arm changes the distribution of support levels, so it is a path comparison under Section 3.5.

Reward distribution. Lean marks the first failing tactic, and tactic-level rewards with first-error propagation outperformed outcome-only rewards in most settings of Kim and Yun [8]. A tactic-level term is heuristic. A reward for valid Lean alone is open to gaming, for example by repeated `try simp`, and a compile-only reward in autoformalization led the model to emit the same simple, irrelevant statement for every problem [9]. Any such term must stay below the value of one accepted proof.

Endpoints and prediction. The primary endpoint is held-out whole-theorem proofs per 1,000 Lean calls. The secondary endpoint is ϕ_{mix} per training step. We predict that the curriculum helps most when the reward is outcome-only. Loop decisions use curriculum validation only, and the test set is scored once. From 0 of 15, an exact McNemar test needs 6 newly solved theorems for $p < 0.05$, so the larger pool with 38 test theorems is the better held-out set.

Preparation. Lean must first replay the 860 linear proofs and every cut point. Then the pass rate at each starting point is measured for the unchanged model.

References

[1] Qwen Team. Qwen2.5 technical report. https://arxiv.org/abs/2412.15115.

[2] A. Hatamizadeh, S. N. Akter, S. Prabhumoye, J. Kautz, et al. RLP: Reinforcement as a pretraining objective. https://arxiv.org/abs/2510.01265.

[3] Hugging Face. TRL v0.24.0 GRPO trainer documentation. https://huggingface.co/docs/trl/v0.24.0/en/grpo_trainer.

[4] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. https://arxiv.org/abs/2306.15626.

[5] S. Polu et al. Formal mathematics statement curriculum learning. https://arxiv.org/abs/2202.01344.

[6] A. Havrilla et al. Teaching large language models to reason with reinforcement learning. https://arxiv.org/abs/2403.04642.

[7] Training large language models for reasoning through reverse curriculum reinforcement learning. https://arxiv.org/abs/2402.05808.

[8] M. Kim and S.-Y. Yun. Process-verified reinforcement learning for theorem proving via Lean. https://arxiv.org/abs/2606.20068.

[9] Y. Huang et al. FormaRL: Enhancing autoformalization with no labeled data. https://arxiv.org/abs/2508.18914.

A Training settings

Table 26 lists the settings from the frozen configs in `configs/`. Every training run used TRL 0.24.0 and learning rate 10^{-6}. "Tokens" is the generated-token cap of the run, not the realized count.

Table 26: Settings of the training runs, read from the configs.

#	Config	G	Max new	Temp.	KL β	Steps
4	`rl/qwen_05b_whole_proof_grpo`	4	256	0.6	0	32 per arm
5	`rl/qwen_05b_prerequisite_grpo`, `_syntax`	4	256	0.6	0	0
6	`rl/qwen_05b_completion_grpo`	4	256	0.6	0	3 per arm
7	`pretrain/qwen_05b_number_theory_rlp`, `_continuation`	4	128	0.8	–	256 each
8	`midtrain/qwen_05b_number_theory_rlp_fp32`	4	128	0.8	–	256
9	`midtrain/qwen_05b_primitive_rlp`	8	512	0.8	–	596
11	`rl/qwen_05b_local_operator_grpo`	8	1,024	0.7	0.02	64
13	`midtrain/qwen_05b_controlled_rlp`	8	1,024	0.8	–	432

The GRPO proof runs of Experiments 4 to 6 sampled with top-p 0.9, and the RLP runs with top-p 0.95. The RLP runs used EMA decay 0.999. Experiments 4 to 6 trained in BF16. Experiments 8, 9, 11, and 13 used FP32 parameters and optimizer moments with BF16 computation.

B Provenance

Table 27 names, for each experiment, the archived record and the raw artifact that holds its counts. "Archived record" paths are in Git commit `8fc8423`. Raw artifacts are on cpu-angad under `formal_rl/` unless the path starts with `artifacts/` in this repository.

Table 27: Sources of the counts.

#	Archived record	Raw artifact	Re-read from raw files
1	`docs/results/01_data_preparation.md`	`artifacts/leandojo_nat_v1/frozen/`; `artifacts/leandojo_number_theory_v2/`	Yes
2	`docs/results/02_qwen_capability_probe.md`	`artifacts/qwen_05b_probe/comparison.json` (local)	Yes

#	Archived record	Raw artifact	Re-read from raw files
3	docs/results/02_qwen_capability_probe.md	artifacts/qwen_05b_instruct_proof/summary.json (local); tensorboard_pilot_20261003/events/	Yes
4	docs/results/03_grpo_proof_pilots.md	artifacts/whole_proof_grpo_20261003/	Yes
5	docs/results/03_grpo_proof_pilots.md	artifacts/prerequisite_rl_v1/	Yes
6	docs/results/03_grpo_proof_pilots.md	artifacts/completion_rl_v1/	Yes
7	docs/results/04_rlp_acquisition.md	artifacts/number_theory_rlp_v1/; artifacts/number_theory_rlp_v2/audit_final/	Yes
8	docs/results/04_rlp_acquisition.md	artifacts/number_theory_rlp_v2/update_audit/; artifacts/number_theory_rlp_fp32_midtraining/	Yes
9	docs/decisions.md, docs/team_workstreams.md	primitive_midtraining_v1/audit_final/diagnosis/	Yes
10	docs/results/05_acquisition_evaluations.md	artifacts/acquisition_eval_v1/; artifacts/decomposition_eval_v2/; artifacts/knowledge_inventory_v1/	Yes
11	docs/results/06_local_action_pilot.md	operator_pilot_v1/gpu_records/out/	Yes
12	docs/results/07_rlp_corpus_audit.md	artifacts/rlp_data_audit_v1/final/	Yes
13	docs/results/08_controlled_rlp.md	artifacts/mathematical_pretraining_v1/; artifacts/controlled_ordered_review_20261006/run.json and ordered_results_v2e/	Yes
PH	This report, Section 9	scripts/audit/classify_proof_failures.py; scripts/audit/curriculum_inventory.py	Yes
AU	This report, Section 10; docs/REWARD_HACKING.md	artifacts/notes/2026-10-10_trainer_checker_report.md; artifacts/notes/2026-10-10_engineering_audit.md; artifacts/notes/2026-10-10_data_build_report.md (all three local); trainer_checker_v1/; nt_false_hyp_v1/; nt_probe/; nt_splits_v3/	No

Architecture values come from the model's config.json and the header of model.safetensors.

Literature claims come from the cited sources, which we read on 8 October 2026.

C Current Lean loop and failure analysis, 13 October

C.1 The current experiment uses Lean during RL

The current comparison implements reinforcement learning with verifiable rewards (RLVR). The supervised parent supplies the starting policy. The policy receives a Lean goal state and generates a tactic block. Lean executes that block from the last verified proof state. An accepted state transition supplies the next observation. When all goals close, the verifier checks the complete original declaration before awarding positive proof reward. The recent completed branches use REINFORCE with a leave-one-out baseline, not PPO. This appendix describes those branches; it does not diagnose every historical checkpoint.

The distinction between a valid step and a complete proof is essential. A step can split one goal into several goals, introduce an unhelpful hypothesis, or reach a state that the policy cannot finish. Such a step establishes execution validity, not mathematical progress. In the current error-feedback mode, a valid changed state receives no positive reward. Complete-proof acceptance receives positive reward. Recognized format, name, type and tactic errors receive heuristic penalties. These penalties classify Lean diagnostics; Lean does not certify their usefulness as a training objective.

The verifier uses Lean `v4.10.0-rc1` with mathlib revision `29dcec074de168ac2bf835a77ef68bbe069194c5`. The source identity receipt reports no mathlib mismatch. A full replay requires a single declaration, target absence, no reported error and a target-specific allowed-axiom report. Closing the interactive goals alone cannot produce a positive proof verdict. The frozen source is `nt_expanded_paired_dev_20261012_v3/code/scripts/train/run_tactic_rl.py`; the final check is `verify_group` in its frozen `bench_lean_verifier.py`.

C.2 Reset and transition contracts are implemented, but fresh checks remain pending

Reset restores the cached root proof-state identifier, the root goals and the set of visited states. It clears the accepted tactic path, repair counter and last diagnostic. A new environment process rebuilds the root if the previous process closes. Each task owns its interactive episode. The source therefore implements episode reset; this source inspection is not a fresh runtime isolation test.

The wrapper commits a new proof-state identifier only after Lean accepts a changed state or reports closed goals. A rejected block leaves the previous verified state and accepted path unchanged. An action can contain several tactics. If a later tactic fails, the wrapper rejects the complete block rather than committing its successful prefix. For permitted repair, the next observation includes the original verified goals and the last diagnostic. The proof path excludes the rejected action.

The complete-proof verifier uses the original environment snapshot for each declaration attempt. A declaration from one completion is therefore not deliberately added to the next completion's verification environment. Timeout and process-exit outcomes remain failures. The source closes the affected process instead of treating missing feedback as acceptance. These contracts address reset, state transition and proof validity. They do not establish that every diagnostic reaches the policy correctly.

Existing integration tests cover goal splitting, reset, rejected-action rollback, repair, multiline actions, forbidden axioms and full replay. Their selected execution is queued after the existing evaluations and targeted Lean reproductions. The tests use the unchanged existing test file against

the frozen learner source. No fresh integration pass is reported here. A failed expectation, skipped check or resource cap must remain visible in the resulting record.

C.3 RL changes intermediate behavior without improving the measured endpoint

Table 28 gives completed native evaluation outcomes. An accepted attempt requires full replay of the original declaration. The denominators count generated attempts, except the final row, which counts distinct validation statements proved with two attempts per statement. DEV selects the supervised parent because neither RL branch proves a broader DEV statement. The composition population is exploratory; it does not establish family-held-out out-of-distribution performance.

Table 28: Completed endpoint outcomes. Unfinished populations remain pending.

Population and metric	Parent	Ordered RL	Shuffled RL
Broader DEV, accepted attempts	0/64	0/64	0/64
Easy retention, accepted attempts	24/48	20/48	21/48
Combined goals, accepted attempts	0/32	0/32	0/32
Component goals, accepted attempts	1/64	3/64	0/64
Full validation, distinct statements	3/392	1/392	Pending

The simple analysis compares the completed parent and ordered validation populations. Table 29 partitions their first actions. Ordered RL produces more valid changed states. Its completed proof count is lower despite those additional transitions. The observation does not establish catastrophic forgetting or a limit of the architecture. It identifies the need to measure completion separately from accepted intermediate actions.

Table 29: First-action outcomes on the same validation population, 784 attempts per model. A stopped attempt can reflect rejection or a resource/format guard.

First-action outcome	Parent	Ordered RL
Stops immediately	739	696
Valid changed state, continues	40	88
Complete accepted proof	5	0

The parent proves three distinct statements; ordered RL proves a different statement. Their proved sets do not overlap in these samples. Ordered RL proves `lean_workbook_6386`, a gcd-divisibility application. It does not reproduce the parent's accepted empty-subset, congruence-addition or gcd/lcm proofs. The corresponding ordered attempts fail with proof-construction, syntax or unresolved-name diagnostics. None of those failures reports opaque exception number four. This rules out that recorded exception as the direct terminal diagnostic for these sampled differences. It does not determine what the model would produce with more samples. The same seed does not align random draws when trajectory lengths differ.

The parent supplies a concrete basic number-theory example:

```
have h := Nat.gcd_mul_lcm a b
simp only [hgcd, hlcm] at h
linarith
```

The native verifier accepts the complete original gcd/lcm statement for this generated path. A separate accepted statement permits the empty set as a witness. Its acceptance is formally valid, but it does not support a claim of substantive number-theory reasoning. The historical tactic screen therefore cannot certify semantic difficulty.

C.4 Repeated failures do not have one tactic as their explanation

The simple analysis detects textual occurrence of `simp_all` in each generated trajectory. It does not establish that the tactic executes or causes the outcome. The parent contains that text in 174 of 784 trajectories. Ordered RL contains it in 311 of 784 trajectories. Table 30 shows descriptive associations for ordered RL.

Table 30: Ordered validation outcomes grouped by generated text. Counts can overlap across outcome columns; this is not a causal comparison.

Generated text	Trajectories	Accepted	Repeated state	Opaque #4
Contains `simp_all`	311	0	4	112
Does not contain `simp_all`	473	1	42	98

Most recorded ordered repeated-state failures occur without `simp_all`. A diagnosis that attributes repetition to this tactic alone is unsupported. The association with opaque exceptions also has uncontrolled alternatives, including task difficulty and other tactics in the same block. A controlled intervention would change the policy's tactic choice while keeping the task and execution conditions fixed. No such intervention is reported.

C.5 Lost diagnostics can change the intended feedback signal

The core REINFORCE source contract uses original sampled token identifiers and disables dropout during sampling and differentiable replay. The loss uses sequence log probabilities and at most one update per sampled group. Groups with zero relative weights skip the optimizer update. The audit does not find a basic leave-one-out formula error. Runtime labels, generation logits and policy divergence were not saved, so this does not verify their numerical values.

Positive relative advantage does not require positive absolute reward. A failed trajectory can receive positive weight if it receives less penalty than the other sampled trajectories. The TRL leave-one-out description explains this baseline. Table 31 counts the exact first diagnostic `internal exception #4` in TRAIN. All such actions receive zero immediate reward. Recognized ordinary tactic errors can receive a negative penalty. The difference can change which failed actions the optimizer favors.

Table 31: Saved TRAIN diagnostic and relative-credit counts. Positive advantage is not positive proof reward.

Action measurement	Ordered	Shuffled
Exact first diagnostic `internal exception #4`	186	183
Those actions with zero immediate reward	186	183
Positive advantage in a group with no proof	98	114
Total generated actions	2007	2047

An opaque exception does not establish a process crash or a model-independent Lean defect. The pinned Lean source defines internal tactic-abort exceptions. The source audit identifies an interface path that can lose the underlying structured diagnostic. The exact runtime exception mapping remains pending. Lost feedback is a candidate training defect; its causal effect on proof completion remains untested. A reward ablation must recover the diagnostic and compare runs from the same parent with fixed tasks, schedules, budgets and endpoint verifier.

C.6 Uniform occurrence does not provide a complete prerequisite curriculum

The ordered and shuffled schedules preserve the same admitted task multiset. Each theorem parent receives equal occurrence. The intermediate application stage covers 9 of 22 parents. The intermediate composition stage covers 11 of 22 parents. Both intermediate stages cover 6 of 22 parents. This leaves many parents without a complete bridge between supplied-answer closure and independent proof construction. The earliest structure tasks include the answer as a hypothesis. Their acceptance demonstrates closure with assistance, not discovering that answer.

Warm supervised prompts match the current step-prompt format and use raw unfenced targets. The recorded warmstart token audit finds no truncation. These observations weaken a generic prompt-format explanation. They do not verify runtime label masks or establish sufficient proof training. The matched frozen-checkpoint diagnostic will compare whole-goal and supplied-answer completion under identical repair settings. Training acceptance itself cannot supply that comparison: the policy changes during training, and the main endpoint uses a different repair budget.

C.7 Proof delivery has a separate serialization defect

The native wrapper preserves the relative indentation of accepted tactic blocks when assembling the full proof. The audited frozen exporter strips leading whitespace before dedenting. This changes the indentation of some multiline bodies. In the saved composition delivery records, both altered multiline bodies fail fresh export replay. Both unchanged single-line bodies pass that saved replay. The source/text audit identifies a downstream delivery defect. It cannot explain the absence of native combined-goal proofs.

Standalone compilation and native REPL verification also use different memory controls. A saved standalone accepted-proof delivery hits its unchanged internal memory limit. An import-only control is required before attributing that failure to the proof. The queued runtime reproductions preserve the original limits and compare original text, exported text, imports and selected accepted demonstrations. No failed file is repaired or counted as accepted by this analysis.

C.8 Evidence boundaries and next checks

The completed evidence supports basic lemma application by some current checkpoints and low independent completion on the evaluated tasks. It supports source-level reset and rollback contracts, a serialization defect and a diagnostic-feedback concern. It does not establish reliable undergraduate-level proof capability, a causal curriculum effect, reward exploitation or proof gain from PPO. Full TEST and shuffled full validation remain unfinished at this snapshot. They are not entered as zero results.

The next checks use existing resources in sequence: full-population assessment, matched whole-versus-assisted assessment, targeted Lean reproductions and the selected existing loop integration tests. The integration tests can falsify the source-level reset, rollback and final-verification interpretation. The matched assessment can reveal sampled deterioration or assistance dependence.

Neither check alone establishes why training changes the policy. A separately controlled training intervention would be needed for that causal question.

C.9 Environment repairs after the source audit

The maintained exporter at `scripts/eval/export_tactic_proofs.py` now passes raw tactic text to the existing indentation-preserving wrapper. Its focused tests reproduce the old multiline failure and then check the replay body, exported file and archive after repair. Rejected actions from a permitted repair remain outside the final proof body. These tests substitute the verifier; they do not establish a fresh Lean replay. The original frozen exporter and recorded outcomes remain unchanged. Future exports must select the maintained command explicitly.

The diagnostic classifier now recognizes the audited expected-term, expected-token, forbidden-tab and unexpected-end parser errors. The existing reward consumer applies its syntax penalty to those recognized cases. Unresolved internal exceptions remain separate. No blanket internal-error penalty or change to the leave-one-out estimator is introduced. The pinned REPL patch at `patches/lean_repl_v4.10.0_rc1_diagnostics.patch` attempts to recover newly logged error messages before conversion discards them. It handles the named tactic-abort exception and preserves other exceptions. Its isolated Lean compilation and original-versus-patched reproduction remain pending. It is not installed in the running evaluation environment.

The future curriculum scheduler rejects missing application or composition starts instead of silently substituting earlier tasks. It rejects application labels that refer to absent, whole-goal or supplied-answer starts. The complete-schedule checks retain uniform parent occurrence and identical ordered/shuffled multisets. This prevents a false completeness claim; it does not construct the missing verified intermediate tasks. The current task package therefore remains incomplete for this stricter curriculum contract. Additional starts require fresh verification before training admission.

C.10 Published small provers guide interface design, not predicted success

Lean-Qwen0.5B uses the same nominal Qwen2.5-0.5B family, supervised adaptation and GRPO. Its RL reward compares generated tactics with reference tactics, and inference uses LeanDojo proof search. That design is a limited precedent for small-model Lean generation, not evidence for our terminal verifier reward. Kimina-Prover-RL-0.6B supplies a near-size RL-trained Lean analogue initialized from a distilled model. Its nominal family name does not imply an exact 500M parameter count. Its larger supervision and search budgets prevent direct comparison with our sampled outcomes. ReProver supplies released Lean 4 tactic-generation and premise-retrieval models. Its supervised retrieval design is a reference for usable mathematical context, not a measured RL contribution to this project.

The LeanDojo interaction implementation separates ordinary Lean errors, completed proofs, proof abandonment and process/protocol crashes. Those distinctions motivate preserving diagnostics rather than treating every rejected request as the same failure. Original proof context and accepted text must also survive replay and export. The project retains its pinned Lean/mathlib pair; adopting an interface contract does not require an untested dependency migration.

C.11 The next causal study remains a separate experiment

A larger study should freeze one shared parent after basics training, then compare RL with the unchanged parent, expert iteration and a shuffled-reward control. Basics improvement belongs to a separate comparison before RL. Use replicated seeds and select checkpoints on DEV only. Plot complete-proof performance against generated tokens at fixed checkpoints. Declare common token,

Lean-call and wall-time ceilings, and report actual usage for every arm. Different algorithms need not exhaust all resource ceilings simultaneously, so equal ceilings alone do not establish equal realized cost. The task map must use the same interactive prompting, repair and verification contract as training. Sampled mixed outcomes provide evidence of solvable tasks, but do not determine the underlying success probability exactly. No larger SFT, task-map sampling or algorithm comparison is launched by the source repairs above.

Reproduction sources. Endpoint counts come from `nt_expanded_paired_dev_20261012_v3/results.json`, `nt_composition_eval_20261012_v1/results.json` and completed validation raw outcomes in `nt_posttraining_eval_20261012_v1/`. The simple saved-record command and controls reside in `nt_deep_diagnosis_20261012_v1/simple_analysis_v1/source.py`, `result.json` and `receipt.json`. Training source and reward findings reside in `training_mechanism.json`, `training_mechanism_warmstart.json` and `training_mechanism_internal_errors.json` under the same deep-audit directory. The loop and export claims reside in `lean_boundary.json`. Runtime checks remain separately owned by `runtime_probe_v1/` and `loop_environment_v1/`. These are remote run-relative paths; infrastructure identifiers are not required to reproduce the interpretation.